INSTITUTIONAL TECHNICAL DOSSIER

# Bloch Protocol

Version 2.0 · 2026

A post-quantum settlement layer moving from proof of work to proof of stake — what has been designed, what has been built, what is actually running, and where each of those three is not the same thing.

**Tiago Beltrão de Azevedo Tenório Acioli**

Licensed AGPL-3.0-or-later · supersedes Edition 1 (Bloch-SIS-PoW, 2026)

**Honest status.**

Unaudited. No third-party security audit has been completed. Genesis-3 is a young, low-hashrate chain and ends by consensus rule at height 39,918; the proof-of-stake node runs on a devnet and has no launch date. Roughly 94% of the carried-over supply is held at one address. BLCH carries no value claim and nothing here is financial, legal or security advice.

**designed ≠ built ≠ booted.**


1 What changed from Edition 1

2 Consensus — proof of work ends, proof of stake begins

3 Economics, supply and governance

4 Cryptography and security

5 Appendix — pre-audit dossier

# What changed from Edition 1

> The text between the rules below is the chapter draft, ready for the Edition 2 manuscript subject to founder review. It is deliberately placed at the front of the dossier, not in an appendix: a reader of Edition 1 must be able to read this one chapter and know exactly which of that document's claims still stand.

---

Edition 1 of this dossier described a protocol named Bloch-SIS-PoW and made four defining claims on its cover: that it was ownerless, post-quantum, pure proof-of-work, and a BlockDAG. One of those four claims survives into this edition. This chapter records what happened to the other three, and to everything downstream of them, because this document's standard — set by Edition 1 itself — is that a dossier whose claims move without a record is worth nothing. Each retraction below names the decision record that made it. Nothing here happened silently.

**1. "Ownerless" has been retracted.** Edition 1's Chapter 4 argued for, and its Chapters 1, 16, and 19 repeatedly relied on, an ownerless commons: no issuer, no foundation, no entity with standing to speak for the protocol or make representations about BLCH. On 2026-08-10 the founder retracted that thesis in writing (ADR-036, which revokes ADR-033 and ADR-034, the founder's relinquishment pact). The reason is stated in the ADR rather than reconstructed here: the Genesis-4 token design allocates 10% of supply for sale to funds, and an allocation sold to investors cannot be sold by nobody — it implies an issuer, and carrying "ownerless" and "issuer" at once would have been false. The replacement structure is the Solana template: a non-profit **Bloch Foundation** (to be created) that holds and distributes the non-founder allocations, signs listing and subscription agreements, and publishes weak-subjectivity checkpoints, beside **Postern Labs Ltda**, the development company, which builds and signs releases and holds no protocol authority. Every sentence in Edition 1 that says "no issuer," "no foundation," "nobody with standing," or "ownerless" is superseded. The governance chapters of this edition (Chapters 11 and 18) describe what exists instead, including the centralisation costs the new structure pays and the controls intended to keep the two-entity split from being letterhead.

**2. "Pure proof-of-work" ends at a known block height.** Edition 1 stated: "no staking, no bonded validator set, and no delegated voting weight standing in for hash power," and presented proof-of-work's externally verifiable cost as half of the project's thesis. That design is being retired. The Genesis-3 chain halts, by consensus rule, at height 39,918; after a pause for review and external audit, **Genesis-4 launches as a proof-of-stake chain** — stake-weighted leader election, a bonded validator set (~25,000 BLCH per validator, post-split), delegation, and slashing (BLOCH-POS-SHA3-LATTICE-MIGRATION.md). The migration design itself names the costs, and this edition repeats them rather than softening them: the merged-mined Bitcoin hashrate that secured Genesis-3 — the cheapest real security this chain will ever have had — is discarded; staking rewards strengthen an investment-contract reading of BLCH; and under proof-of-stake, coins are consensus weight, which makes the concentration described in point 6 below a consensus-security fact rather than only an economic one. What the migration buys is also stated: deterministic finality, deletion of an entire class of consensus

bugs the PoW chain actually suffered (order-dependent difficulty validation), and the end of the chain's dependence on rented, departing hashrate.

**3. Finality: Edition 1's account is inverted.** Edition 1's Chapter 8 was titled "Finality & the Dropped FFG" and said, accurately at the time, that a Casper-FFG-style finality overlay "was never integrated, never activated, and never shipped," and that the live finality story was proof-of-work depth plus checkpoints. In Genesis-4 the design position is the opposite: a Casper-style finality rule — **Bloch-BFT**, two voting rounds, a ≥ 2/3 stake quorum, checkpoints at epoch boundaries, surround- and double-vote slashing — is the core of the consensus design, carried by the same post-quantum signatures as everything else (no BLS aggregation exists for lattice schemes, and the design pays that bandwidth cost rather than adding a classical primitive). Its status is **built, not booted**: it runs on an internal devnet and nowhere else. Chapter 9 documents it.

**4. The chain Edition 1 described is now history — and it stops.** Edition 1 documented Genesis-2 as the live chain. Since then: Genesis-3 relaunched the network on 2026-07-29 (fresh height 0, SHA-256d with little-endian target comparison from genesis, the carryover as an opening balance, merged mining with Bitcoin from height 8,500); and Genesis-3 itself ends at **height 50,000**, by a consensus rule, planned for roughly mid-August 2026 (the height is fixed; the date moves with hashrate). At that height a signed snapshot of every balance becomes the canonical record; mining income ends; there is then a period of months with **no running chain** while the proof-of-stake implementation is reviewed and audited; and Genesis-4 opens with the snapshot balances. There is no claim process, no swap, and no migration transaction — a balance crosses by doing nothing, and anything that asks otherwise is an attempt at theft. Chapters 6 and 7 give the full account. Readers of Edition 1 should note what this means for its text: the BlockDAG and GhostDAG ordering described in its Chapter 6 are retired with proof-of-work; Genesis-4 is a linear chain.

**5. The economics are replaced, not amended.** Edition 1's Chapter 16 described a 21-billion nominal supply that was explicitly not hard-capped (a perpetual tail subsidy), a Bitcoin-style halving schedule, and a founder premine of 17% locked behind a 10-year cliff and 40-year vest, presented as "structurally passive." None of that describes Genesis-4. The supply is a fixed **100,000,000,000 BLCH**, produced from the 21 B nominal by a pure ×100/21 redenomination (every balance and every allocation scaled by the same factor — more units, identical shares, no new money), and the cap is enforced as a **consensus invariant**: cumulative issuance is a committed component of the state, and a block that would exceed the cap is invalid. Stated at its true strength and no stronger: no in-protocol mechanism — no vote, no key, no governance path — can raise it; a hard fork adopted by every operator can change any rule of any chain. Emission goes to validators over 40 years on a 10% annual disinflation curve (there is no halving), and the allocation is: carryover 17.97% (liquid at genesis), founder grant 10% (10-year cliff, then 40-year linear vest), VC 10%, team 10%, marketing 4%, liquidity 5%, validator emission 43.03%. The 17% locked premine of Edition 1 **does not exist in Genesis-4**; the founder's position is instead the sum of a new, strictly-vested 10% grant and a carried-over mined balance that is liquid — which is the subject of the next point.

**6. Concentration is now treated as a first-class fact. Edition 1 did not treat it.** Edition 1 disclosed the locked premine at length but was silent on the distribution of the circulating, mined supply. The measured numbers: approximately **94% of the supply crossing the Genesis-3 snapshot sits at one address, the founder's** — post-split, 17,046,829,380 BLCH, or 70.5% of circulating supply at slot 0 — and by decision of 2026-08-11 that balance crosses

liquid and stakeable, on the same terms as every other carried balance, because it was mined under the same rules as every other balance. Under proof-of-stake this is not a footnote; it is the consensus. If that balance is staked, independent participants cannot reach the migration's distribution gate from emission alone — the gate moves only when coins change hands. The migration design therefore makes distribution a hard Go/No-Go condition (gates G1–G4: independent stake ≥ 15% of circulating supply, no entity above 25% of active stake, Nakamoto coefficient ≥ 7, ≥ 200 validators), measured while excluding stake whose beneficial owner is the founder, the Foundation, or Postern Labs — and if the gates are not met, the transition does not activate. Chapter 17 carries the year-by-year arithmetic; nothing in it is flattering, and it is published anyway, because the alternative is that someone else publishes it first.

**7. The securities posture is weaker, and this edition says so.** Edition 1 wrote: "BLCH is not a security and not an investment asset … no token sale, no listing effort, and no price or value claim made on its behalf by anyone with standing to make one, because nobody has that standing." That defence rested on the ownerless structure and has been retracted with it. There is now an identifiable issuer, a planned sale to funds, and staking yield — facts that move the question toward the centre of the investment-contract test, not away from it. This edition does not assert a legal conclusion in either direction: the question is under legal review that the migration plan classifies as blocking, and until it concludes, the only honest statements are that no value claim is made, nothing in this document is investment advice, and the old "nobody has standing" argument is no longer available.

**8. The name and the mark.** "Bloch-SIS-PoW" named the protocol by its proof-of-work construction; with that construction retiring, the protocol is now **Bloch Protocol**. The visual identity changes with it: Edition 1's navy-and-triangle cover is replaced by the white-ground, emerald (#0E6E5A) identity whose mark is the **Bloch sphere** — the protocol is named for Felix Bloch, and the sphere is the canonical representation of a qubit, which is the object this protocol's cryptography is built to survive (docs/site/BRAND-KIT.md). A name is not a thesis, but a renamed document owes its readers the note.

**What did not change.** The cryptography — the hybrid ML-DSA-65 ‖ Falcon-1024 signature suite in which both algorithms must verify, the PQClean-derived implementations, the frozen-dependency posture — is carried into Genesis-4 unchanged, and is an explicit fixed input of the migration design. The harvest-now-decrypt-later argument of Edition 1's Chapter 3 stands word for word. The Coherence shielded pool's frozen formats are preserved byte-for-byte (and the pool is provably empty on this mainnet, so the transition moves no shielded value). The license is AGPL-3.0-or-later. And the discipline this chapter itself obeys — **designed ≠ built ≠ booted**, every capability labelled with the state it has actually reached — is retained from Edition 1 in full, because it was the best thing in that document, and it is the reason this chapter could be written at all.

---

*End of Chapter 2 draft.*

---

# Consensus — the end of proof of work and the shape of proof of stake

## Bloch Institutional Dossier — Edition 2

### The Consensus Chapters: Genesis-3, Its Planned End, and the Proof-of-Stake Design

```
Document:   ED2-CONSENSUS
Edition:    2 · 2026
Supersedes: Edition 1, Chapters 5–8 (Consensus I — Proof of Work;
            Consensus II — The BlockDAG & GhostDAG; The Genesis-2 Hard Fork;
            Finality & the Dropped FFG)
Repository: bloch-pos (gitlab.com/blochsispow-group/bloch-pos)
License:    AGPL-3.0-or-later
```

**Honest status, inherited and enforced.** Edition 1 adopted a mantra and this edition keeps it: **designed ≠ built ≠ booted**. Having a specification, having code that implements it, and having that code running as the consensus-enforced rule on a live network are three different states, and every capability in these chapters is labelled with which of the three it has reached. Conflating them is the single most common way a technical dossier misleads a reader, and Edition 1 was written to avoid it systematically. So is this one — which matters more this time, because Edition 2 must document a *designed* consensus (proof of stake) alongside a *booted* one (Genesis-3), and must also document a reversal: a finality mechanism Edition 1 correctly described as dropped is now the center of the design. That reversal is explained in Chapter 4, honestly, including what changed in the reasoning and what has *not* changed in the facts.

Three status vocabularies used throughout:

- **Designed** — specified in a reviewed document; no claim that code exists.
- **Built** — implemented and tested as code; no claim that it runs anywhere that matters.
- **Booted** — running as an enforced rule on a network; the only state that constitutes a property of a live chain. "Booted (devnet)" means running only on local, throwaway-key validator processes — explicitly *not* a public network.

---

# 1. Genesis-3 — the consensus that runs today

Edition 1 documented Genesis-2, the live chain of mid-2026. On 2026-07-29 the network relaunched as **Genesis-3**: not a fork of Genesis-2 but a brand-new chain starting at height 0 with its own genesis block and a distinct chain identifier, ingesting the same carried-over ledger as opening balances. This chapter documents what Genesis-3 actually runs — all of it booted, on mainnet, today — and Chapter 2 documents the unusual fact that distinguishes this edition from every ordinary protocol dossier: the chain has a consensus-encoded final block, and it is days away.

## 1.1 Chain identity and lineage

Genesis-3 is registered as `ChainId::Genesis3Mainnet = 0xB10C_0004` (`crates/bloch-crypto/src/core/mod.rs:179`). The registry comment is explicit about what it is and is not: "a brand-new chain that starts at height 0 with its OWN genesis block (distinct coinbase banner ⇒ distinct genesis hash — NOT a fork of Genesis-2), ingests the SAME carry-over ledger as Genesis-2 as its opening balances" (`mod.rs:170–178`). The chain-id is folded into every transaction sighash for replay domain separation (`mod.rs:139–151`), so a Genesis-2 signature cannot be replayed on Genesis-3 or vice versa.

The carryover is not optional equipment. `chain_requires_carryover` (`mod.rs:396–411`) makes the snapshot flag mandatory at node start for Genesis-3, for a fail-closed reason stated at the match arm: a node started without it "would produce a chain with the right rules and an empty ledger — every balance silently gone, and nothing in the protocol objecting" (`mod.rs:400–405`).

**Status: booted.** Genesis-3 mainnet has been producing blocks since 2026-07-29.

## 1.2 SHA-256d, ASIC-native from height zero

The proof-of-work function is SHA-256d over an 80-byte Bitcoin-layout mining header, exactly as Edition 1 described for Genesis-2. The chain-id → PoW mapping is a single exhaustive function, `pow_algorithm` (`mod.rs:199–207`), with `ChainId::Genesis3Mainnet => PowAlgorithm::Sha256d` (`mod.rs:205`) — miner and validator both route through it, so they provably agree.

One defect Edition 1 spent a chapter on is structurally absent here. Genesis-2 needed a height-gated hard fork (`SHA256D_LE_FORK_HEIGHT = 2400`, `mod.rs:2473`) to correct a big-endian target comparison that rejected all standard ASIC work. Genesis-3 validates little-endian **from height 0**: `sha256d_le_fork_height_for(ChainId::Genesis3Mainnet)` returns `0` (`mod.rs:2489–2496`), with the rationale in place — "a fresh chain has no legacy big-endian blocks to grandfather, so ASIC shares validate from block 1" (`mod.rs:2485–2488`). The endianness lesson of Edition 1's Chapter 7 was paid for once and encoded permanently.

**Status: booted.**

## 1.3 Difficulty — and the incident that rewrote how it is validated

The target block interval is 30 seconds (`TARGET_BLOCK_TIME`, `mod.rs:262`, sourced from `tokenomics_v2.rs:82`), with a Bitcoin-style retarget every 60 blocks (`GENESIS3_RETARGET_WINDOW = 60`, `mod.rs:2281`, compile-time asserted equal to the Genesis-2 window at `mod.rs:2288`).

Edition 1 described difficulty retargeting as "a purely mechanical, deterministic adjustment computed identically by every full node from the same on-chain history." On Genesis-3 that sentence was, for a period, false in a way this dossier is obliged to record. The legacy validation path derived expected difficulty from **node-local mutable state** — a `current_bits` meta key rewritten on every accepted block, and a timestamp column keyed by height alone, which in a DAG is last-write-wins. Two nodes running an *identical binary* could disagree purely because they accepted blocks in a different order, and every follower froze permanently at the first retarget boundary where its cache had diverged. The consequence is stated in the code, not

softened: "The chain effectively had one producer and no independent validator" (`mod.rs:32–44`).

The fix is `DIFFICULTY_ANCESTRY_FORK_HEIGHT = 30_030` (`mod.rs:125`): at and above that height, expected bits are a pure function of the block's own selected-parent ancestry, computed at one height-gated choke point shared by the validator and every template producer, with ancestry-incomplete cases failing closed (`mod.rs:102–111`). The 90-line comment above the constant preserves the full post-mortem, including a first activation attempt that was itself wrong (a distant flag-day at 30,000 "is not a safety margin at all: it is a guarantee that no follower ever reaches the fix", `mod.rs:56–65`) and an emergency intermediate raise after a halt at height 28,080 where the producer's own template and its own validator disagreed (`mod.rs:72–100`).

**Status: booted** (active above 30,030 on the live chain).

> **Note.** This incident matters beyond its mechanics, and it recurs in Chapters 3 and 4. It is the clearest evidence in the project's record for two claims this edition relies on: first, that Genesis-3's practical security posture was weaker than "young PoW chain" already implies — for stretches of its life the network had one block producer and no independent validation; second, that consensus values derived from node-local mutable state are how identical honest nodes split. The proof-of-stake design in Chapter 3 treats the second claim as a design axiom (`crates/bloch-pos-committee/src/transition.rs:14–29`).

## 1.4 GhostDAG ordering

Genesis-3 remains a BlockDAG ordered by GhostDAG, as Edition 1's Chapter 6 described: blocks may cite multiple parents, and the canonical total order is computed over the blue set by accumulated blue work rather than by a single longest chain. The anticone parameter is `GHOSTDAG_K = 10` (`mod.rs:251`). Everything in Edition 1's structural account — tips, blue/red classification, `blue_score` as the DAG analogue of height, parallel-block absorption instead of orphaning — carries over unchanged and is not re-derived here.

One measured fact is worth adding: with a 30-second target and DAG parallelism, the observed cadence at the time of writing is faster than target — 21.57 s/block measured on 2026-08-12 (commit `38258aa`, branch `deploy/g3-terminal-50000`). The DAG absorbs the parallel production, exactly as the structure is designed to.

**Status: booted.**

## 1.5 Merged mining with Bitcoin (AuxPoW)

Since Edition 1, Genesis-3 gained merged mining: because Bloch's PoW is SHA-256d exactly like Bitcoin's, the same hash can secure both chains. A miner hashes a Bitcoin block whose coinbase carries a commitment to a Bloch block; if that single hash meets Bloch's (lower) target, an AuxPoW proof lets Bloch accept the block — "no extra hashing, no new ASICs" (`crates/bloch-crypto/src/core/auxpow.rs:1–17`). The verifier is faithful to the standard Bitcoin/Namecoin `CAuxPow` format — parent-header PoW against Bloch's `bits`, coinbase proven at index 0 in the parent merkle root, and the `fa be 6d 6d` merge-mining marker

carrying the aux root (`auxpow.rs:10–17`) — so existing merge-mining pool tooling applies. Bloch's merged-mining chain id is `AUXPOW_CHAIN_ID = 0x0B10` (`auxpow.rs:36`).

Activation followed the project's established flag-day idiom: the verifier shipped inert with the activation height at `u64::MAX`, then a coordinated fleet upgrade set `AUXPOW_ACTIVATION_HEIGHT = 8500`, activated 2026-08-01 (`mod.rs:16–22`). A block carrying an `auxpow` below the height is invalid, fail-closed; the validation arm is height-gated at `mod.rs:1756`.

The honest caveat ships in the module documentation and this dossier repeats it rather than paraphrasing it away: "merged mining only secures Bloch with the FRACTION of BTC hashrate that opts in, and lets a large BTC miner attack at ~zero marginal cost — for a young chain this can worsen the 51% risk, not fix it. It is a bootstrap lever, not a security guarantee" (`auxpow.rs:23–26`).

**Status: booted** (active at and above height 8,500; merged blocks accepted on mainnet since 2026-08-01).

### 1.6 What Genesis-3 is, in one table

| COMPONENT | MECHANISM | SOURCE | STATUS |
|---|---|---|---|
| Chain identity | `0xB10C_0004`, fresh genesis, carryover opening balances | `mod.rs:179`, `mod.rs:396–411` | Booted |
| Proof of work | SHA-256d, 80-byte Bitcoin-layout header, little-endian from height 0 | `mod.rs:199–207`, `mod.rs:2489–2496` | Booted |
| Difficulty | 30 s target, 60-block retarget; ancestry-derived bits above h30,030 | `mod.rs:262`, `mod.rs:2281`, `mod.rs:125` | Booted |
| Ordering | GhostDAG over a BlockDAG, K = 10 | `mod.rs:251` | Booted |
| Merged mining | Namecoin-style AuxPoW under Bitcoin, active at h8,500 | `auxpow.rs`, `mod.rs:22`, `mod.rs:1756` | Booted |
| Finality gadget | None — PoW depth only, exactly as Edition 1 stated for Genesis-2 | Ch. 4 below | n/a |
| Terminal height | Chain ends at a fixed height; blocks above are consensus-invalid | Ch. 2 below | Built; flag-day deploying |

## 2. The terminal height — why Genesis-3 ends

This is the chapter Edition 1 could not have contained. Genesis-3 has a consensus-encoded last block. At that height a signed balance snapshot is taken; the snapshot — not the chain — becomes the canonical record; and Genesis-4, a proof-of-stake chain, launches from that artifact roughly six months later. Nothing about this is framed as an upgrade of the running chain. The chain is not being continued. It is being ended, deliberately, and the protocol's own constants say so.

### 2.1 The rule

`GENESIS3_TERMINAL_HEIGHT` is the last valid height on the Genesis-3 mainnet; blocks **above** it are invalid (`mod.rs:437–438`). The terminal height itself is valid — "it is the last block, and the height the snapshot is taken at" (`is_past_terminal_height`, `mod.rs:457–466`). The per-chain lookup is exhaustive with no wildcard arm, the same fail-closed idiom used for the carryover requirement: adding a chain-id without deciding whether it terminates is a compile error, "instead of silently inheriting 'runs forever'" (`mod.rs:440–455`). A unit test pins that only Genesis-3 terminates (`mod.rs:3064–3071`) and another pins that the terminal block itself remains valid (`mod.rs:3074–3086`).

The rule is wired at every point where a block could be born or accepted:

- **Block acceptance** rejects anything past the terminal height before any other validation work: "this chain has ended; Genesis-4 launches from the snapshot" (`src/main.rs:2537–2544`).
- **The internal miner** stops assembling rounds and idles rather than grinding PoW its own node would reject (`src/main.rs:1820–1835`).
- **Stratum V1** template production refuses past-terminal templates (`src/stratum/session.rs:102`), and **Stratum V2** likewise (`src/stratum_v2/template_adapter.rs:55`) — so ASICs are never handed work on a chain that has ended.

## 2.2 The value: 80,000, then 50,000 — a flag day with a three-day fuse

The constant was introduced at 80,000 on 2026-08-11 (commit `ced885d`; the value on this repository's integration branch, `mod.rs:438`). On 2026-08-12 the founder lowered it to **50,000**, shipped as a single-constant diff on the dedicated branch `deploy/g3-terminal-50000` (commit `38258aa`) against the exact tree already running fleet-wide — the commit message states the discipline plainly: a three-day flag day is not the place to ship a month of work with no mainnet soak. At the measured 21.57 s/block cadence, height 39,918 arrives roughly three days from that commit, and every node must be running the 50,000 binary before it does. The commit is equally plain about the failure mode: "A node still on the 80,000 binary keeps accepting blocks above 39,918, and the moment one does, the halt itself becomes the fork."

The number 50,000 was originally argued from a carryover-cap race (`docs/specs/BLOCH-TOKENOMICS-V4.md` §3.1) that has since been retired along with the cap itself (§3: every holder carries over in full, no scale-down, no founder-exclusion list). What survives of the rationale is the notice logic: about two weeks of public notice at decision time, on a round number — and the spec's counterintuitive but correct observation that *longer* notice is worse here, because holders need do nothing (balances are captured on-chain automatically) while the one action notice enables is accumulating more coins before the cut (§3.1).

**Status: designed (V4 §3.1–3.2); built and tested (`mod.rs:437–466` and the four wiring sites above); booted-in-progress — the 50,000 flag-day binary was cut 2026-08-12 for fleet deployment inside the ~3-day window. The deploy window is itself the named risk.**

## 2.3 Why the chain ends: the honest argument, in full

Edition 1 said, in its Chapter 5 and again in Chapter 8, that Genesis-2 was "a young chain with low accumulated hashrate, and is realistically 51%-attackable," that how long that window would persist was "an open question," and that the dossier would not pretend to answer it with a number it did not have. Edition 2 can now answer it: the window did not close. It was not closing. Genesis-3 inherited the same condition — low external hashrate, concentrated block production, and (per §1.3 above) stretches with effectively one producer and no independent validator. Merged mining, activated in August, borrowed real Bitcoin work but by its own module's admission is "a bootstrap lever, not a security guarantee" (`auxpow.rs:26`) — and can worsen the 51% profile against a large Bitcoin miner, not improve it.

A protocol in that condition has three honest options: continue and keep labelling confirmations as reversible indefinitely; wait for organic hashrate that two chain generations failed to attract; or end the chain deliberately and move its ledger to a security model whose budget does not depend on external hashrate arriving. The project chose the third. The specific reasoning for *halting* rather than running the old chain in parallel is in `BLOCH-TOKENOMICS-V4.md` §3.2 and is worth restating because it is not obvious:

1. **A halt must be a consensus rule, not an announcement.** "A chain does not stop because it was announced." If blocks above the terminal height were merely unwanted, miners would keep producing them and the halt would be "a fork nobody agreed to. Making them **invalid** is what actually ends the chain" (`mod.rs:419–423`, V4 §3.2.1). This is a flag day in reverse, with every flag-day hazard this project has already lived through intact.
2. **A running dead chain is worse than a stopped one.** Had Genesis-3 kept producing during the ~six months between snapshot and Genesis-4 launch, coins would be mined by people receiving nothing in the successor, and the rational miner switches off the day after the snapshot anyway — leaving the network without hashrate during exactly the months it still has users, wallets and an explorer pointed at it (V4 §3.2).

## 2.4 After the halt, the chain's history stops being evidence

This is the non-obvious consequence, and the code comment above the constant states it with no varnish: "Once mining stops, this chain's history stops being evidence. PoW security is bought with ongoing hashrate; with none, rewriting history below the terminal height costs almost nothing" (`mod.rs:431–433`). Anyone with modest SHA-256d hashrate can, months after the halt, produce an alternative chain ending at 39,918 with different balances — and it may even carry more accumulated work than the real one (V4 §3.2.2).

Therefore **the signed snapshot artifact is canonical, not the chain**. At the terminal height the balance set is produced, hashed, signed, and its digest published widely enough that it cannot be quietly replaced — the same pattern already used for `carryover.tsv.gz` and its `.sha256` companion. The snapshot digest is to be embedded in the Genesis-4 genesis block itself, "precisely so the record does not depend on a chain nobody is defending" (`mod.rs:433–435`, V4 §3.2.2). Un-upgraded miners who continue past 50,000 continue on a fork; that is tolerable "only because the canonical artifact is the signed snapshot at this height, not whatever chain has the most work afterwards" (`mod.rs:427–430`).

> **Risk.** Between the halt and the Genesis-4 launch there is no live chain defending this ledger — only a signed artifact and the breadth of its publication. The security of every carried-over balance during that window is exactly the security of that digest's distribution

> and of the signing key. This is a real, named trust interval, not a technicality, and no mechanism in this chapter removes it. It is the price of ending a chain whose work-based security was not, in honest terms, doing the securing.

---

# 3. Proof of stake — the Genesis-4 design

Genesis-4 (design codename **Bell**) replaces proof-of-work with proof of stake, SHA-3 hashing, and deterministic finality, while keeping two things fixed by founder constraint: the hybrid ML-DSA-65 ‖ Falcon-1024 post-quantum signature suite, unchanged, in every consensus role; and the Coherence ZK ledger with its frozen SHAKE-256 formats (`docs/specs/BLOCH-POS-SHA3-LATTICE-MIGRATION.md`, header block). The design document's own status line is reproduced here rather than improved upon: "DRAFT — design only, not approved, not scheduled."

The design is not only a document. A pure consensus crate (`crates/bloch-pos-committee`) implements it rule by rule, and a devnet node binary (`crates/bloch-pos-node`) composes those rules into running validator processes. Chapter 5 draws the built/booted line precisely; this chapter describes the design and cites the code that exists for each part.

## 3.1 Shape: a linear chain of slots and epochs — GhostDAG retired

Time is divided into **30-second slots**, 32 slots to an epoch (≈16 minutes) (`SLOT_DURATION_SECS = 30`, `SLOTS_PER_EPOCH = 32`, `crates/bloch-pos-committee/src/params.rs:29-34`). The slot duration is deliberately identical to today's PoW block target "so the transition adds no new propagation pressure" (`params.rs:32-33`). One validator is designated proposer per slot by stake-weighted public sortition over a hash-based beacon (§3.5); with one proposer per slot, concurrent block production is an anomaly rather than the norm, and **GhostDAG is retired**: Genesis-4 is a linear chain. The header's `parents` vector narrows to a single parent under a consensus rule `parents.len() == 1`, and fork choice becomes LMD-GHOST over attestation weight rather than blue score (migration spec §5.2).

The V4 block header removes `bits`, `nonce`, and `timestamp` (derived from the slot) — and the spec is explicit about what removing `bits` buys: it "deletes the order-dependent difficulty bug class" that split Genesis-3's mainnet (spec §5.3, §2; the incident of §1.3 above). The ≈4.6 KB hybrid proposer signature sits in the block envelope, not the header, so the signed header stays 248 bytes and header-sync paths never carry the signature (spec §5.3).

**Status: designed (spec §5); header/envelope built (`crates/bloch-pos-committee/src/header.rs`); booted (devnet only).**

## 3.2 Committees: the active set partitioned, not sampled

Every epoch, the full active validator set is shuffled deterministically and **partitioned** into 32 committees, one per slot — every active validator lands in exactly one committee and votes exactly once per epoch (`crates/bloch-pos-committee/src/committees.rs:1-35`). The union

of an epoch's committees *is* the active set, so the finality quorum denominator is total active stake, with no sampling variance and no ambiguity.

This is a correction the project's own adversarial review forced, and the module documentation preserves the defect rather than deleting it. The first design sampled a 128-validator committee, and review finding F1 showed the quorum denominator had no coherent reading — against network stake, a 128-member sample cannot hold two-thirds once the network exceeds ~192 validators, so finality would be structurally unreachable; against committee stake, sampling variance lets a ~30% adversary exceed one-third of the committee roughly one epoch in five and stall finality below the nominal threshold (`committees.rs:5–18`). Finding F2 was worse in a quieter way: independent per-slot draws routinely selected the same validator into several slots of one epoch, whose duplicate same-target attestations the slashing logic then correctly flagged — "honest validators slashed themselves" (`committees.rs:29–35`). Partitioning removes both by construction.

The cost is carried honestly: one ≈4,589-byte hybrid signature per active validator per epoch, which is cheaper than the sampled design below 384 validators (where the launch gates put the set) and imposes a scaling ceiling around ~4,096 validators, at which point sub-sampling — and finding F1 with it — would have to return unless signature aggregation becomes practical; measurement says that is "research, not engineering" (`committees.rs:37–46`, `spikes/prover-cost/RESULTS.md`).

The partition for epoch N is seeded with a one-epoch look-ahead (the beacon mix fixed at the close of epoch N−2), closing a grinding attack in which the proposers of an epoch's trailing slots could withhold reveals to re-sort the next epoch's partition (finding F6, `committees.rs:60–80`).

The epoch transition derives the partition rather than storing it — "a stored partition is a cache" — and asserts it covers the eligible set exactly once (`crates/bloch-pos-committee/src/transition.rs:1264–1273`).

> **Note — residual constants.** `params.rs` still carries `COMMITTEE_SIZE = 128` and `SLOT_SUBCOMMITTEE_SIZE = 8` (`params.rs:17,27`), and the `finality.rs` module docstring still narrates the earlier "full epoch committee of 128" design (`finality.rs:5–8`). The spec's §5.1 table marks both constants **removed**, and the composition layer binds the partition path (`transition.rs:1143`, `transition.rs:1268–1269`, `committees::epoch_committees` at `committees.rs:171`). The stale constants and docstring are documentation debt, recorded here so no reader mistakes them for the operative design.

**Status: designed (spec §5.1, §6.5.3); built with the F1/F2/F6 corrections (`committees.rs`, `sample.rs`, `transition.rs`); booted (devnet only).**

## 3.3 Fork choice: LMD-GHOST

Between finalized checkpoints, the canonical head is selected by LMD-GHOST — Latest Message Driven: only each validator's most recent attestation counts, the weight of a block is the total effective stake of validators whose latest message is that block or a descendant, and the head is found by walking from the latest **justified** checkpoint, taking the heaviest child at

each step (`crates/bloch-pos-committee/src/forkchoice.rs:1-10`). Equivocating validators are dropped from fork-choice weight permanently (`forkchoice.rs:54-58`). Starting the walk at the justified checkpoint gives two properties for free: finalized history can never be reorganized out, and the walk is bounded by the unfinalized suffix (`bloch-pos-node/src/engine.rs:50-53`).

The devnet engine binds this rule and the documentation states why the naive alternative is not merely weaker but wrong: longest-valid-chain "lets a proposer with no attested support drag the chain by building fast … Length is not the security statement in proof of stake; attested stake is" (`engine.rs:33-48`). The engine rebuilds the fork-choice store from scratch on every head computation rather than caching it — the `expected_bits` lesson applied where a cache would be cheapest to get wrong (`engine.rs:54-60`).

**Status: designed; built; booted (devnet only) — the devnet engine selects its head via `forkchoice_head` (`engine.rs:370-374`).**

## 3.4 Finality: Casper-style justification and finalization

One checkpoint per epoch. A checkpoint is **justified** when attestations carrying it as target — from that epoch's committee members, with the currently highest justified checkpoint as source — account for at least two-thirds of active stake; the comparison is integer arithmetic, `3·attesting ≥ 2·total` in `u128`, so "exactly 2/3 justifies and 2/3 − 1 satoshi does not, identically on every node" (`crates/bloch-pos-committee/src/finality.rs:15-20`, `committees.rs:273`). A checkpoint is **finalized** by consecutive justification: when the supermajority link `source → target` has `target.epoch == source.epoch + 1`, the source becomes final (`finality.rs:21-24`).

The safety argument fits in a paragraph and the module carries it: each validator counts at most once per epoch (duplicates deduped; *conflicting* votes mark the validator an equivocator and count for no target, order-independently), so two conflicting checkpoints at one epoch would need two disjoint ≥2/3 quorums out of one whole — impossible; at most one root can be justified per epoch, hence at most one finalized (`finality.rs:56-62`). Requiring one uniform source per justification is what makes each justification a property of exactly one link rather than a mosaic (`finality.rs:31-40`).

The state is a pure fold over the vote history — no clock, no cache, "no channel to anything but its inputs" — and the incremental epoch processor is tested to equal the from-history fold (`finality.rs:42-54`). The module names its ancestor incident: consensus state living outside committed inputs is what froze every Genesis-3 follower on 2026-08-08.

**Status: designed (spec §5.1, §6.5); built (`finality.rs`); booted (devnet only) — devnet validator processes justify and finalize over a local mesh (`bloch-pos-node/src/main.rs:5-8`). No checkpoint has ever been finalized on any public Bloch network.**

## 3.5 Randomness: hash-based commit-reveal, no VRF

Proposer sortition and committee shuffling are seeded by a RANDAO-style accumulator over SHAKE-256 preimages: each validator commits a hash chain at registration (`RANDAO_CHAIN_LENGTH = 8_192`, `params.rs:53`), reveals one preimage per slot it proposes, and the beacon mixes reveals per slot under a dedicated domain tag (`DS_RANDAO`,

`params.rs:83`; `crates/bloch-pos-committee/src/beacon.rs`). The design uses commit-reveal rather than a VRF because no standardized post-quantum VRF exists and the founder constraint forbids new primitives (spec §6.3–6.4); uniqueness comes from preimage binding, not signature uniqueness. Every signed or hashed object in the protocol lives under a fixed 16-byte domain-separation tag so no digest can be replayed across domains (`params.rs:69–119`).

**Status: designed; built (`beacon.rs`, `sample.rs`); booted (devnet only).**

## 3.6 Slots, stakes, and the lifecycle

The staking lifecycle is specified in spec §7 and implemented in `crates/bloch-pos-committee/src/staking.rs`:

| PARAMETER | VALUE | SOURCE |
|---|---|---|
| Minimum deposit | 25,000 BLCH | `staking.rs:97` |
| Activation delay | 8 epochs (≈2.1 h), rate-limited at 4 activations/epoch | `staking.rs:103,108` |
| Exit delay | 32 epochs (≈8.5 h) | `staking.rs:113` |
| Withdrawal delay | 2,048 epochs (≈22.8 days) | `staking.rs:120` |
| Per-validator stake cap | 1% of active stake, fixed-point | `delegation.rs` (spec §5.1) |
| Slashing | proposer equivocation and Casper surround/double votes | `slashing.rs`, `attestation.rs` |
| Inactivity leak | quadratic after 4 epochs of non-finality, quotient 64 | `params.rs:59,67` |

The **inactivity leak** deserves its own sentence because it is the liveness backstop deterministic finality requires: after four epochs without finality the stall is presumed a partition or abandonment rather than jitter, and absent validators bleed stake quadratically until the remaining live stake is again a supermajority of the shrunken total — sized for recovery in hours, not days (`params.rs:54–67`, `finality.rs:25–29`).

**Weak subjectivity** is confronted rather than waved at. Under PoW a fresh node needs only the genesis; under PoS, once stake has cleared withdrawal its keys can sign an alternative history at zero cost, and a fresh node cannot distinguish the two from inside the protocol. The answer is a weak-subjectivity checkpoint obtained out of band, adopted as a floor the node never reverts below (`crates/bloch-pos-committee/src/ws.rs:1–19`, `docs/specs/BLOCH-WEAK-SUBJECTIVITY.md`) — the module's own documentation records that the spec's first window arithmetic was wrong and re-derives it (`ws.rs:21–25`). **Status: designed and built as format/verification rules; the fresh-sync path that would use them is not built (§5.2).**

## 3.7 The genesis cohort and the one-third rule

A fresh PoS genesis has a bootstrap circularity: deposits need blocks, blocks need validators, and there is no PoW to seed the set. Genesis-4 therefore launches with a **genesis validator cohort** the founder funds and operates — "a centralised start, by construction" (`crates/bloch-pos-committee/src/genesis_cohort.rs:7–10`) — under a consensus rule, not

a promise: the cohort is a fixed set published in the genesis block, it can only shrink, and its combined effective stake is capped by a schedule declining linearly from the whole set at genesis to **one-third at one year**, then holding (`genesis_cohort.rs:30–40`). One-third is not chosen for modesty: it is exactly the share that can stall a two-thirds quorum, so the rule says in one number that after year one the founder cannot halt the chain alone. The module states with equal clarity what the rule does *not* give: one-third is the liveness threshold, not the safety one — nothing about the cap stops finalization of a bad state, which needs 2/3 and "is out of reach either way" (`genesis_cohort.rs:16–27`). Stake above the cap earns nothing and carries no weight; it is not confiscated (`genesis_cohort.rs:37–40`).

**Status: designed (V4 §3.3); built (`genesis_cohort.rs`); booted (devnet only).**

### 3.8 The state-transition discipline

Every rule above composes in one place: `Transition::apply_block` / `process_epoch` (`crates/bloch-pos-committee/src/transition.rs:1–11`), written against three rules that are Genesis-3's failures generalized into axioms: every consensus value comes from the parent's committed state, never from node-local mutable state; no function may depend on input arrival order; and caps are measured against totals they do not themselves reduce (`transition.rs:14–30`). The committed state is a plain value — no clock, no cache, no interior mutability — and even the state root is recomputed from leaves on every call, "because a memoized root that survives one forgotten invalidation is exactly how `expected_bits` split the mainnet on 2026-08-08" (`transition.rs:15–21`).

**Status: built (with the largest test surface in the crate); booted (devnet only).**

---

## 4. The retraction — finality, dropped and readopted

Edition 1's Chapter 8 stated, as plainly as its FACTS record allowed: "Casper FFG was never integrated, never activated, and never shipped on Genesis-2. There is no bonded validator committee running on the live chain… Any scaffold or reference code that touches on FFG concepts that may exist in the repository is unshipped research." And its Chapter 5 gave the design philosophy behind the drop: "a bonded validator set is, definitionally, a privileged operator class whose stake or membership confers a role no other node can perform."

Genesis-4's core is a Casper-style finality gadget. This section explains the inversion — what changed in the reasoning, not merely in the outcome — because a dossier that reversed a position this central without accounting for it would forfeit the credibility the designed/built/booted discipline exists to earn.

### 4.1 What has not changed: the facts Edition 1 stated remain true

Start with what is *not* being retracted. Every factual sentence in Edition 1's Chapter 8 remains true today, and remains true of Genesis-3: no finality gadget has ever finalized a block on any public Bloch network. Genesis-3's finality is PoW depth, probabilistic, Bitcoin-style, exactly as

Chapter 8 described for Genesis-2, and it will remain so until the chain's last block at height 39,918. The FFG-era research scaffolds still exist in this repository and still carry their honest labels: `crates/bloch-ffg` — a static 14-of-21 committee design — says in its module header "**Status: FOUNDATION. NOT wired into consensus.** Standalone + tests only… Unaudited" (`crates/bloch-ffg/src/lib.rs:18-20`), and a later miner-weighted FFG-BFT overlay concept was likewise explicitly dropped (recorded in the migration spec's baseline table: "Finality: PoW depth only. FFG-BFT explicitly dropped", spec §2). Three finality designs have now existed in this project's history; zero have booted on a public network. The Genesis-4 gadget of Chapter 3 is **designed and built, booted only on a local devnet** — and this edition will not describe it in any stronger terms until a public network finalizes a checkpoint under it.

What is being retracted is a *thesis*, not a record: the thesis that pure proof-of-work depth was the right — and sufficient — finality model for this chain.

## 4.2 The reasoning Edition 1 gave, and where the record broke it

Edition 1's argument for pure PoW had two legs.

**Leg one: probabilistic finality backed by accumulated work suffices, and matures.** Chapter 5 was honest that Genesis-2 was young and 51%-attackable, and framed that as the transient condition of "every young proof-of-work chain… before enough independent hashrate accumulates on it," with the duration "an open question that depends on external adoption." That framing contained an unexamined assumption: that the accumulation would happen. The record since is the answer. Two chain generations, an ASIC-native relaunch, standard Stratum infrastructure, and merged mining with Bitcoin did not produce an independent hashrate base. The difficulty post-mortem in the code concedes that for stretches "the chain effectively had one producer and no independent validator" (`mod.rs:43-44`); the AuxPoW module concedes that borrowed Bitcoin work "can worsen the 51% risk, not fix it" (`auxpow.rs:25-26`); and the terminal-height comment concedes the endpoint of the logic: "PoW security is bought with ongoing hashrate; with none, rewriting history… costs almost nothing" (`mod.rs:431-433`). A finality model whose guarantee is "the cost of reversion rises with accumulated work" provides, on a chain where work never meaningfully accumulated, approximately nothing — and Edition 1's own risk boxes said as much in the conditional. Edition 2's difference is only that the conditional has resolved. Waiting longer was not a strategy; it was the absence of one.

**Leg two: a bonded validator set is a privileged operator class, incompatible with ownerlessness.** This argument was correct as stated and remains correct. What changed is the honesty of its application. The ownerless-PoW framing implicitly compared PoS-with-a-committee against PoW-as-Bitcoin — an open set of miners disciplined by real external cost. The PoW that Bloch actually had was, for measured stretches, one producer and no independent validator (`mod.rs:43-44`), on a chain whose supply was ~94% held by one address (spec §0.1). That is a privileged operator class in everything but name — worse than a bonded committee in one respect, because nothing on-chain named it, capped it, or scheduled its decline. The proof-of-stake design does not solve the concentration; the migration spec opens by stating it as the plan's "single largest risk" and refuses to launch without distribution gates (spec §0, §11). But it converts an unnamed de facto privilege into named, consensus-bounded ones: the genesis cohort is published in the genesis block, can only shrink, and its weight is forced below the finality-stalling threshold within a year by rule

(`genesis_cohort.rs:30–40`). Between an undeclared monopoly of work and a declared, capped, decaying monopoly of stake, the second is the more honest object — and honesty about who holds power is the property this project's governance story actually rests on.

There is a third, smaller strand: Edition 1 treated "no committee" as protecting the validity/finality distinction. The PoS design keeps that distinction fully intact — validity remains deterministic and proof-gated (now against committed state under `transition.rs`'s rules), while finality becomes a *stronger* claim than depth: accountable, slashable, two-thirds of bonded stake. Nothing in the Chapter 8 taxonomy is abandoned; the finality column upgrades from "probabilistic, cost-based" to "deterministic, accountable" — on a chain whose cost-based column had, in practice, no cost in it.

## 4.3 What the reversal pays, stated without discount

The migration spec's §0 lists the three prices before the architecture, and this dossier repeats them in the same order, unhedged:

1. **Stake concentration.** ≈94% of existing supply at one address, measured at height 18,809 (spec §0.1). "On day one of a naive PoS migration, the founder *is* the consensus." Mitigations — the genesis-cohort cap, the 1% per-validator cap, Nakamoto-coefficient launch gates — bound it; none solves it. Distribution is a hard Go/No-Go launch gate (spec §11), and this edition will treat any Genesis-4 launch that skipped those gates as a defect in the record, not a footnote.
2. **Securities framing.** Staking rewards to bonded holders materially strengthen an investment-contract reading of BLCH; the civic-node, value-beside-the-point posture "survives PoW far more comfortably than it survives PoS." Legal review is a Phase-0 blocking deliverable (spec §0.2).
3. **A discarded security budget.** Merged mining is the cheapest real security Bloch will ever have, and PoS discards it, together with the miner-facing go-to-market (spec §0.3). Chapter 2's terminal-height argument is why the budget was judged insufficient anyway; this line records that it was a real thing given up, not a nothing.

> **Note.** The shortest honest summary of this chapter: Edition 1 said "no finality gadget exists, and pure PoW is the design." Both halves were true. Edition 2 says "no finality gadget has ever booted on a public network, and pure PoW is no longer the design — because the empirical premise under the old design, that work would accumulate, failed." The first half is still a fact; the second is a decision, taken with its costs written down.

---

# 5. What is not ready

This chapter is the edition's spine, because everything in Chapter 3 could be read — wrongly — as a description of a running network. The Genesis-4 node exists at **devnet stage**, and its own binary says so before this dossier does.

## 5.1 What actually runs

The devnet is real and non-trivial: N validator processes produce blocks, attest, justify and finalize over a local TCP mesh, with real ML-DSA-65 ‖ Falcon-1024 hybrid signatures and append-only block-log persistence where restart equals deterministic replay (`crates/bloch-pos-node/src/main.rs:5-8`). The engine is a single consensus thread driven by the slot timer and network events, binding the `Transition` / `CommittedState` seam and LMD-GHOST head selection (`engine.rs:1-6`, `engine.rs:370-374`); a node that rejects its own produced block panics loudly, a rule named after the Genesis-3 h28,080 incident (`engine.rs:24-30`). Devnet keys are throwaway by construction, never printed, 0600 on disk; production keys are out of scope for this tool entirely (`main.rs:17-22`).

## 5.2 What does not exist — the node's own list

The binary's module header is the authoritative inventory, quoted rather than paraphrased (`main.rs:10-15`):

> "What it is NOT yet — honestly, per the integration plan (`docs/specs/BLOCH-POS-NODE-INTEGRATION.md`): no RocksDB store, no libp2p gossip, no transactions (deposits/exits/transfers), no slashing-evidence pipeline, no weak-subjectivity sync, no RPC, no fork choice beyond a linear chain, no mainnet genesis manifest."

Spelled out, because each item is a load-bearing absence:

- **No transactions.** The devnet chain carries consensus objects only. The staking lifecycle of §3.6 is built as validation rules (`staking.rs`), but no deposit, exit, transfer, or Coherence operation can be submitted to, carried by, or executed on the devnet chain. A chain that cannot move value is a consensus test rig, and this one is labelled as such.
- **No libp2p.** Networking is a localhost TCP full mesh with length-prefixed frames — "This is not the production network layer" (`net.rs:1-7`). The plan is to adapt the Genesis-3 libp2p/gossipsub stack with the 2026-08-07 mesh fixes; "that work is not done" (`net.rs:5-7`). The pure crate's gossip admission-control layer (`gossip.rs`) exists but is not wired to this mesh (`net.rs:12-14`).
- **No RPC.** There is no interface for a wallet, explorer, or exchange to query or submit anything. The RPC surface is specified (`docs/specs/BLOCH-RPC-V4.md`) — designed, not built.
- **No slashing-evidence pipeline.** Slashing conditions are built as rules (`slashing.rs`); the machinery that would detect, package, and carry evidence on-chain is not.
- **No weak-subjectivity sync.** The checkpoint format and verification exist (`ws.rs`); the fresh-node sync path that would consume them does not.
- **No mainnet genesis manifest.** There is no Genesis-4 genesis. The input it requires — the signed terminal snapshot of Chapter 2 — does not exist yet either.

One entry on the list has been overtaken by the code and is recorded as such: the engine now implements LMD-GHOST fork choice with reorganization (`engine.rs:33-48`), so "no fork choice beyond a linear chain" overstates the gap; the header list simply lags the engine. Two further pieces of recorded documentation debt: the pure crate ships two parallel producer/validator seams (`transition.rs` vs `derive.rs` / `produce.rs`) whose committed state roots are not byte-compatible — the engine binds exactly one and the unreconciled

situation is flagged in place rather than papered over (`engine.rs:8–22`); and the residual sampled-committee constants noted in §3.2 (`params.rs:17,27`).

## 5.3 No launch date

There is no Genesis-4 launch date, and this dossier does not imply one. The schedule that exists is relative and conditional: the snapshot at height 39,918 (taken 2026-08-13), then Genesis-4 "roughly six months later, after code review" (V4 §3.2) — with the migration spec's Go/No-Go gates (§11) standing between the code and any mainnet, distribution gates included, and the spec itself still marked "not approved, not scheduled." Six months is a planning figure, not a commitment; the honest ordering is: terminal snapshot, then the integration work of §5.2, then third-party review, then gates, then — only if the gates pass — a launch.

## 5.4 The consensus stack, labelled — the table this edition exists for

| CAPABILITY | DESIGNED | BUILT | BOOTED |
|---|---|---|---|
| Genesis-3: SHA-256d PoW, GhostDAG K=10, 30 s target | ✓ | ✓ | ✓ mainnet |
| Genesis-3: AuxPoW merged mining under Bitcoin | ✓ | ✓ | ✓ mainnet (h≥8,500) |
| Genesis-3: ancestry-derived difficulty | ✓ | ✓ | ✓ mainnet (h≥30,030) |
| Genesis-3: terminal height 39,918 | ✓ (V4 §3.1–3.2) | ✓ (`mod.rs:437–466` + 4 wiring sites) | flag-day deploying (commit `38258aa`) |
| Signed terminal snapshot artifact | ✓ (V4 §3.2.2) | tooling staged (`deploy/artifacts/bloch-snapshot-utxo`) | — (taken at h39,918) |
| G4: slots/epochs, linear chain, V4 header | ✓ | ✓ | devnet only |
| G4: partitioned committees, sortition, RANDAO beacon | ✓ | ✓ | devnet only |
| G4: LMD-GHOST fork choice | ✓ | ✓ | devnet only |
| G4: Casper-style justification/finalization | ✓ | ✓ | devnet only |
| G4: inactivity leak | ✓ | ✓ | devnet only |
| G4: slashing rules | ✓ | ✓ | devnet only (no evidence pipeline) |
| G4: staking lifecycle (deposits/exits/withdrawals) | ✓ | ✓ (rules) | ✕ (no transactions exist) |
| G4: genesis cohort, one-third-in-one-year cap | ✓ | ✓ | devnet only |
| G4: weak-subjectivity checkpoints | ✓ | ✓ (format/verify) | ✕ (no sync path) |

| CAPABILITY | DESIGNED | BUILT | BOOTED |
|---|---|---|---|
| G4: transactions, libp2p, RPC, RocksDB, mainnet genesis | partly (specs) | × | × |
| Any finality gadget on a public Bloch network | — | — | **never, to date** |

> **Risk, closing.** The consensus that is booted is ending by rule; the consensus that would replace it is a devnet that cannot yet carry a transaction. Between them stand a signed artifact, roughly six months of integration and review, and launch gates that are allowed to say no. That is the honest state of Bloch consensus as of this edition, and every stronger sentence a reader may encounter elsewhere should be checked against the table above.

---

*Copyright © 2026. Licensed AGPL-3.0-or-later. This chapter cites code at specific lines of the `bloch-pos` repository (integration branch, and where noted the `deploy/g3-terminal-50000` branch) and of the Genesis-3 node tree (`src/`); line numbers drift as code moves, but every claim above was verified against the cited line at the time of writing.*

# Economics, supply and governance

## Bloch — Institutional Dossier, Edition 2

### Economics & Governance

```
Document:    ED2-ECONOMICS-GOVERNANCE
Replaces:    Edition 1 chapters 4 (The Ownerless Commons),
             16 (Economics & Tokenomics), 19 (Governance & Roadmap)
Status:      DRAFT for Edition 2 assembly — describes designs and code that
             are NOT live; the running chain today is Genesis-3 (PoW),
             scheduled to halt at a terminal height
Sources:     docs/adr/ADR-036-retract-ownerless-adopt-foundation.md
             docs/adr/ADR-034-founder-anonymization-relinquishment-pact.md
             docs/specs/BLOCH-ENTITY-STRUCTURE.md
             docs/specs/BLOCH-TOKENOMICS-V4.md
             docs/specs/BLOCH-POS-SHA3-LATTICE-MIGRATION.md (§4, gates)
             crates/bloch-pos-committee/src/tokenomics_v4.rs
             crates/bloch-pos-committee/src/staking.rs
             crates/bloch-pos-committee/src/delegation.rs
             crates/bloch-pos-committee/src/genesis_cohort.rs
Created:     2026-08-12
```

**Reading rule, inherited from Edition 1 and applied without exception: designed ≠ built ≠ booted.** Everything in this chapter that concerns Genesis-4 — the Foundation, the allocations, the vesting, the emission curve, the validator bond, delegation — is at most *built*: constants, state transitions, and tests exist in a standalone crate that the running node does not reference (`crates/bloch-pos-committee/src/tokenomics_v4.rs` states this in its module header: "Nothing here is active"). None of it is booted. The entity structure is at most *designed*:

`docs/specs/BLOCH-ENTITY-STRUCTURE.md` is marked DRAFT, and the Bloch Foundation does not exist as a legal person at the time of writing.

**Source-of-truth rule.** This chapter does not restate consensus constants; it cites the files that define them. Where a specification's prose and a compiled constant disagree — and at the time of writing they do in at least one place (the carryover total in `BLOCH-TOKENOMICS-V4.md` §1–§2 differs in its last five digits from `CARRYOVER_TOTAL_BLOCH` in `tokenomics_v4.rs`) — **the compiled constant governs**, because it is pinned by compile-time assertions that fail the build if the arithmetic drifts, and prose is pinned by nothing.

---

## 1. The retraction of the ownerless commons

### 1.1 What Edition 1 asserted

Edition 1 did not mention ownerlessness in passing; it made it the load-bearing premise of the entire document. Chapter 4 ("The Ownerless Commons") asserted, as structural properties and not marketing posture:

- that the protocol has **no owner, no foundation, and no official company site** presenting it as a corporate product;
- that there was **no token sale** — "public, private, or otherwise" — no listing effort, and no market-making arrangement (Chapter 16.1);
- that there is **no issuer, no promoter, and no counterparty** "who could make representations about BLCH's value even if they wished to" (Chapter 16.1, Note);
- that Postern Labs is a builder *on* the protocol in exactly the sense a wallet vendor builds on Bitcoin, with no special standing (Chapter 4.3);
- that "no foundation collects fees, controls a treasury on participants' behalf, or issues binding protocol directives" (Chapter 4.4);
- that there is "no legal entity chartered to steward the protocol, hold a treasury on its behalf, fund development on its behalf, or speak for it" (Chapter 19.2).

These statements were true when written, of the chain they described. They are **retracted** as descriptions of the project going forward.

### 1.2 What changed, and where it is recorded

**ADR-036** (`docs/adr/ADR-036-retract-ownerless-adopt-foundation.md`, accepted 2026-08-10) retracts the ownerless thesis in writing. It formally retracts ADR-033 (the decentralisation model that restored the ownerless position) and **ADR-034** — the founder anonymisation and relinquishment pact, which had committed the founder to withdrawing from all forward-facing materials and retaining no governance or operational authority from launch. ADR-036's own words: "ADR-033 and ADR-034 no longer describe the project."

What replaced them:

- **A sponsoring organisation and an issuer.** The governance model adopted is Solana's two-entity split: a non-profit **Bloch Foundation** (to be created) holding and distributing the non-founder allocations and stewarding the network, beside **Postern Labs Ltda**, the Brazilian

development company that builds the node, the OS, the apps, and the wallet (`docs/specs/BLOCH-ENTITY-STRUCTURE.md` §2).

- **A VC allocation.** Tokenomics V4 allocates 10% of supply for sale to crypto funds, with the Foundation as counterparty of the round (`BLOCH-TOKENOMICS-V4.md` §7B; `VC_BLOCH` in `tokenomics_v4.rs`). At the time of writing this is a *planned* sale — an allocation and a vesting schedule exist in code; no round has closed.
- **A founder grant** replacing ADR-034's relinquishment: a new 10%-of-supply grant under a 10-year cliff and 40-year linear vest (`FOUNDER_BLOCH`, `founder_vested_sat` in `tokenomics_v4.rs`), on top of the founder's carried-over balance. ADR-036 is explicit that this "should be described as what it is."

### 1.3 Why it changed

ADR-036 gives the reasons plainly, and they are worth repeating because they are reasons, not rationalisations:

1. **Tokenomics V4 is not compatible with ownerlessness.** A 10% allocation sold to funds introduces investors with a return expectation and, in practice, an issuer. Carrying both positions at once — an ownerless thesis and a VC round — was flagged internally as an inconsistency that had to be resolved in writing before anything was published. The retraction is that resolution.
2. **Weak subjectivity needs a signer.** The PoS migration design identified "who signs the checkpoint in an ownerless system?" as its sharpest unresolved conflict. With a Foundation there is an answer — the Foundation publishes weak-subjectivity checkpoints. ADR-036 calls this what it is: "a real centralisation cost, honestly stated — and it is a cost the ownerless design could not pay at all."
3. **Listing needs a counterparty.** Exchange integration was blocked in part because no legal person existed to sign an agreement. Now one will. (Post-quantum custody remains a separate, harder blocker, unaffected by any of this.)

### 1.4 What the retraction does *not* retract

Retracting the ownerless thesis does not silently convert the protocol into an administered system, and the distinction matters for the governance chapter (§4 below):

- **Consensus rules still change only by operator adoption.** No transaction variant, key, vote, or governance path inside the protocol can alter the supply cap or any other consensus rule; the cap is a `const` with no setter (`TOTAL_SUPPLY_BLOCH` documentation in `tokenomics_v4.rs`). A hard fork adopted by every operator can change any rule — that was true under the ownerless thesis and remains true; "impossible to change" was never claimed and is not claimed now.
- **The codebase remains AGPL-3.0-or-later**, and authorship remains disclosed.
- **ADR-034's honest-scope section was already candid** that the founder retained economic weight ("anonymity ≠ absence of stake") and that entities persisted. What is retracted is the relinquishment of control and the anonymisation — not the honesty norms of that document, which this Edition keeps.

### 1.5 The securities register, restated for Edition 2

Edition 1 stated: "BLCH is not a security and is not marketed as an asset," and could support that statement with three facts — no sale, no issuer, no listing effort. **Those three supporting facts are gone**, and Edition 2 does not pretend otherwise. ADR-036 records the consequence without euphemism: "Selling to investors with a return expectation, plus staking yield, plus an identifiable issuer and promoter, is close to the centre of the investment-contract test rather than the edge of it." The Phase-0 legal review is therefore classified as *blocking*, not precautionary.

What this dossier can still say, and says: **this document makes no value claim, contains no offer or solicitation, sets no price target, and does not market BLCH as an investment asset.** That register — factual description with no promise of value — is retained from Edition 1 in full. What is no longer said is that no one *could* make such representations: an issuer will exist, and whether the planned sale makes BLCH a security in any given jurisdiction is exactly the question the blocking legal review exists to answer. This dossier does not pre-judge that answer in either direction.

### 1.6 Why the retraction is written as a retraction

A document that quietly rewrites its Chapter 4 invites the accusation that it would quietly rewrite anything. Edition 1's credibility rested on disclosure discipline — the same discipline now requires stating that the central governance claim of Edition 1 was made, was meant, and was withdrawn, with the ADR trail (`ADR-033` → `ADR-034` → `ADR-036`) left intact and citable. The history is kept because auditors read history.

---

## 2. Tokenomics V4

Source of record: `docs/specs/BLOCH-TOKENOMICS-V4.md` (design and decisions) and `crates/bloch-pos-committee/src/tokenomics_v4.rs` (the constants, schedules, and compile-time invariants). Status: **built, not booted** — the constants take effect only if and when a Genesis-4 chain is produced from them.

### 2.1 The 100,000,000,000 figure is a redenomination, not new supply

On 2026-08-12 total supply moved from the 21 B nominal to 100 B as a **pure split of exactly 100/21** (`SPLIT_NUMERATOR` / `SPLIT_DENOMINATOR` in `tokenomics_v4.rs`). Every bucket scales by the same ratio; every percentage is unchanged; **nobody is diluted**. The code's own comment states the reading rule: "any text that reads it as 'more supply for holders' is wrong." This is not asserted on trust — `tokenomics_v4.rs` carries a compile-time assertion per bucket proving `new × 21 == old × 100` exactly; if any bucket were ever scaled by a different ratio, that would be a dilution, and the build would fail.

Three costs of the split are recorded rather than absorbed (see also `docs/audit/CERTIK-PRE-AUDIT-DOSSIER.md`, 2026-08-12 entry):

- the supply in satoshis no longer fits the Go SDK's signed `int64`; the SDK must migrate (asserted in `tokenomics_v4.rs` so it cannot be forgotten);
- per-address conversion cannot be exact, because 100/21 never divides a power of ten — `split_g3_sat` truncates, and the ceremony must state its dust rule and close the accounting against the pinned total;

- the emission curve was re-derived, since scaling the old one overshot the allocation.

### 2.2 Allocations

The buckets, each named by its constant in `tokenomics_v4.rs` (shares are ratios and therefore survive the split unchanged; the BLCH figures live in the file, not here):

| BUCKET | CONSTANT | SHARE | UNLOCK |
|---|---|---|---|
| Carryover (the whole Genesis-3 ledger) | `CARRYOVER_TOTAL_BLOCH` | 17.97% | Liquid at genesis |
| Founder — new grant | `FOUNDER_BLOCH` | 10% | 10-yr cliff, 40-yr linear (`founder_vested_sat`) |
| VC / crypto funds | `VC_BLOCH` | 10% | 12-mo cliff, 24-mo linear (`vc_vested_sat`) |
| Development team | `TEAM_BLOCH` | 10% | 18-mo cliff, 36-mo linear (`team_vested_sat`) |
| Marketing | `MARKETING_BLOCH` | 4% | 25% at genesis, rest over 24 mo (`marketing_vested_sat`) |
| Liquidity | `LIQUIDITY_BLOCH` | 5% | 100% liquid at genesis (`liquidity_vested_sat`) |
| Validators | `VALIDATOR_EMISSION_BLOCH` | 43.03% | Emitted over 40 years — never in anyone's custody |

A compile-time assertion pins the sum to `TOTAL_SUPPLY_BLOCH`. The Foundation holds the VC, team, marketing, and liquidity buckets — 29% of supply (`FOUNDATION_HELD_BLOCH`) — making it the largest single holder for the first decade (`BLOCH-ENTITY-STRUCTURE.md` §3). The founder's total position is the carried-over balance plus the new grant: 27.04% of supply (`FOUNDER_TOTAL_BLOCH`, pinned by assertion). The grant was cut from 17% to 10% on 2026-08-11 with the difference reallocated to validators — the only reallocation to date that moved supply *away* from an insider bucket (`BLOCH-TOKENOMICS-V4.md` §1).

The carryover crosses as **one balance set, with no founder line, no taint list, and no holder cap** (founder decision 2026-08-11; the retired cap is kept as a named zero, `HOLDER_CARRYOVER_CAP_BLOCH`, so stale code fails loudly). Those coins were mined on the same chain under the same rules as everyone else's; and liquid includes **stakeable** — see §3, where the consequences are stated in numbers.

### 2.3 Vesting

Schedules are consensus-enforced unlock functions, not promises: each bucket has a `*_vested_sat(slot)` function in `tokenomics_v4.rs`, and §8.2 of the spec sets the standard — "a vesting schedule that lives in a spreadsheet is not a vesting schedule." Cliffs are deliberately staggered (VC at 12 months, team at 18, founder at 120) so no two buckets begin unlocking in the same month — the "cliff wall" is the most cited failure mode in vesting design. The founder's vest is linear **per slot**, not in monthly tranches: a step function would create hundreds of scheduled moments where a block of stake becomes spendable at once, each a visible, game-able date.

The founder grant's 10-year cliff and 40-year vest is far beyond any market benchmark, and deliberately so — the carried-over balance arrives liquid, so the grant is the part of the founder's position that can still be made to wait. It is not, however, the whole position, and

Edition 1's description of the founder allocation as "structurally passive" **does not carry over**: it was true of a design in which the founder's entire position sat behind the cliff, and it is not true of V4, where 16.89% of supply is liquid at slot 0. That sentence from Edition 1 Chapter 16.3 must not be quoted as if it described V4.

### 2.4 Emission

The validator allocation is emitted over 40 years (`EMISSION_YEARS`, `EMISSION_SLOTS` in `tokenomics_v4.rs`). The spec adopts **smooth disinflation at 10% per year** (`BLOCH-TOKENOMICS-V4.md` §6.1): year-one issuance is 4.36–4.37% of total supply, declining every year, summing to the allocation minus an irreducible dust residual (`EMISSION_DUST_SAT` — under the allocation, never over; an earlier claim of a zero residual was arithmetically impossible and is corrected in the file rather than repeated). The code provides three curves (`validator_reward_flat_sat`, `validator_reward_halving_sat`, `validator_reward_decay_sat`) with the decay curve marked recommended; the spec records the 10% decay as decided. No curve is yet wired into a running consensus — that is a Genesis-4 act.

Two register rules the spec imposes on any public figure, kept here:

- **The denominator is load-bearing.** Inflation figures are issuance over *total* supply (how Solana and Ethereum report). Against *circulating* supply the same curve reads over 100% in year one, purely because almost everything is still vesting at genesis. Any published figure must name its denominator (`annual_inflation_bps` in `tokenomics_v4.rs`).
- **"Maximum ever issued", not "total supply."** Base fees are 50% burned during the emission era (§6.3.2), so circulating supply never reaches the cap; the two figures diverge from the first burned fee onward.

The emission curve, not the vesting schedule, is what decides whether the concentration gates can ever be met — the spec's month-by-month model (§7A) shows a flat curve failing gate G2 for roughly five years while a front-loaded curve clears it from month 24. That is why the curve is treated as the most consequential economic parameter in V4.

### 2.5 The cap as a consensus invariant

V4 is hard-capped, reversing V2's perpetual tail — a reversal recorded as such (`BLOCH-TOKENOMICS-V4.md` §6.2 flags that the tail existed to fund security after emission, and that an ADR retracting the tail rationale is still owed). The cap is not a documentation figure: cumulative issued supply is a committed component of the state root (`state_root::TAG_ISSUED_SUPPLY`), and every node refuses a block whose committed issuance exceeds `TOTAL_SUPPLY_BLOCH` (`TransitionError::SupplyCapExceeded`). The honest strength of the claim, quoted from the constant's own documentation: no mechanism *inside* the protocol can raise the cap — no transaction variant, no key, no vote, no governance path — and a hard fork adopted by every operator can change any rule, this one included, "so 'impossible to change' would be false and is not claimed."

The fee-only end state inherits the question the tail existed to avoid: after year 40 the entire security budget is fee revenue, and there is a named revenue cliff at the era boundary (§6.3.2). Whether fees suffice by then is unknowable now; that it must be monitored long before then is knowable and stated.

---

## 3. Concentration

Edition 1 did not have this chapter. Edition 2 has it because the numbers exist, they are measurable by anyone, and a dossier that leaves them for an auditor to discover has chosen its readers' side against itself.

### 3.1 The measurement

Measured on Genesis-3 at height 39,918 and carried under the split (`tokenomics_v4.rs`, `LARGEST_CARRYOVER_ADDRESS_BLOCH` and `CARRYOVER_TOTAL_BLOCH`, both pinned by assertion): the largest single address holds **~94% of the carryover** — which is itself the entire circulating float apart from the Foundation's liquid tranche. The founder's carried-over balance is **~70.5% of everything circulating at slot 0**, and the Foundation's liquid holding is the other 25.0% — two holders account for the whole genesis float (`BLOCH-TOKENOMICS-V4.md` §4A, §7B). A re-measurement at the Genesis-3 terminal height (50,000, decided 2026-08-12; was 80,000) will re-pin both constants; until then the h39,918 figures govern.

### 3.2 If the founder stakes: the §4A.1 arithmetic

`BLOCH-TOKENOMICS-V4.md` §4A.1 works the consequences of the 2026-08-11 decision that a carried-over balance that is liquid is also stakeable — a decision pinned by tests (`staking.rs::carryover_liquid_balance_is_stakeable`, `tests/committee.rs::carryover_liquid_balance_delegates_as_stake`) so it cannot be reverted silently. The arithmetic, which should be read before anyone quotes a decentralisation date as a forecast:

- **Staked at genesis, the founder's balance is ~94% of active stake — a Nakamoto coefficient of 1.** Gate G2 (largest entity < 25% of active stake) fails outright, and because rewards are pro-rata to stake, the shares are *conserved* under compounding: the figure does not decay with time.
- **Gate G1 is unreachable from emission alone if the founder stakes.** G1 requires independent eligible stake ≥ 15% of circulating supply (`BLOCH-POS-SHA3-LATTICE-MIGRATION.md` §4). Share conservation holds the independent share of active stake at its starting point — 6.05% — at every horizon, and active stake can never exceed circulating supply. Under this scenario G1 is "not late, it is unreachable — not at year five, not at year forty — from emission alone. The only thing that moves G1 is coins changing hands" (§4A.1).
- **If the founder abstains**, the earliest arithmetic G1 crossing is a bound of about month 9 — a bound, not a forecast, since the founder-operated genesis cohort earns much of the early emission.

The honest summary the spec itself draws, adopted here as the Edition 2 position: the stakeability decision removed the last consensus-enforced distinction between the founder's coins and anyone else's, and in exchange **the gates stop being a schedule and become a measurement of behaviour** — whether G1–G4 are ever met is decided by whether the founder's carryover stakes and whether coins actually distribute, and consensus constrains neither.

### 3.3 The three mechanisms, and what each one does not reach

Three consensus mechanisms bound concentration. Each is real; each has a stated limit; **none can see beneficial ownership**, because no on-chain metric can see who stands behind an

address.

**The genesis-cohort declining cap** (`crates/bloch-pos-committee/src/genesis_cohort.rs`). The chain starts with a founder-funded, founder-operated validator cohort — there is no other option on a fresh genesis with no PoW phase (`BLOCH-TOKENOMICS-V4.md` §3.3). The cohort is a fixed set published in the genesis block, shrink-only, and its combined weight is capped by a linear taper from 100% at genesis to one third at one year (`cohort_cap_bps`), where it holds. One third is chosen because it is the finality-*stall* threshold: below it the founder cannot halt the chain alone. What it does not say: one third is the liveness threshold, not the safety one (two thirds), and — stated in the module's own docs — nothing prevents funding *new* validators after genesis under addresses outside the cohort. Past the cohort, the one-third figure is a commitment verified externally, not a rule.

**The 1% per-validator cap** (`crates/bloch-pos-committee/src/delegation.rs`, `MAX_VALIDATOR_STAKE_BPS`, resolved by fixed-point iteration over a fixed round count (`CAP_FIXPOINT_ROUNDS`) rather than against the uncapped total, precisely so the cap's strength does not degrade as concentration rises). What it does not reach: it is Sybil-bypassable by splitting stake across validators, per its own documentation, which is why §4A.1 treats share conservation, not the cap, as the conservative assumption.

**The churn limit** (`delegation.rs`, `WARMUP_RATE_BPS` with the `MIN_CHURN_SAT` floor and partial activation; history in `BLOCH-POS-STAKE-CHURN.md`). At most a small fraction of active stake activates per epoch, so a takeover requires publicly visible queue traffic over many epochs rather than one block. What it does not reach: it slows entry; it does not distinguish whose stake is entering, and it cannot.

An earlier fourth mechanism — taint-based ineligibility of founder coins — **no longer exists**: with the single-set carryover decision there is no class of coin to mark, and no provenance criterion survives anywhere in the admission path (`CARRYOVER_TOTAL_BLOCH` docs in `tokenomics_v4.rs`). The spec's earlier delegation rule "tainted coins cannot delegate" is stale against that decision. The dissolution cuts both ways and both are recorded: an unaudited list-writing power ceased to exist, and a consensus-enforced exclusion of founder stake ceased to exist with it.

### 3.4 The gates, and who is not counted

The activation gates (`BLOCH-POS-SHA3-LATTICE-MIGRATION.md` §4): G1 — independent eligible stake ≥ 15% of circulating; G2 — no entity above 25% of active stake, top-3 below 50%; G3 — Nakamoto coefficient ≥ 7; G4 — ≥ 200 active validators, ≥ 50 unaffiliated with Postern Labs. The metrics that compute G2/G3 (`Registry::top_share_bps`, `Registry::nakamoto_coefficient`, with the Nakamoto coefficient computed at the one-third threshold, not one half, which would flatter it) measure the **operator** view — what consensus sees. Foundation stake delegated across forty operators reads as forty independent participants; a genesis cohort of 64 founder-funded records reads as 64. Therefore the reporting rule, written down before the numbers become convenient (`BLOCH-ENTITY-STRUCTURE.md` §5.1, extended to the genesis cohort in `BLOCH-TOKENOMICS-V4.md` §3.3): **stake whose beneficial owner is the Foundation, the founder, or Postern Labs does not count toward G1–G4.** That is a reporting rule, not a protocol rule, and it is labelled as such.

The launch statement Edition 2 adopts, verbatim from the spec: **the chain starts centralised, by construction, and the gates measure the distance from there.**

## 4. Governance today

### 4.1 The two entities

Per `docs/specs/BLOCH-ENTITY-STRUCTURE.md` (DRAFT — structure proposed, jurisdiction and board not decided):

| | BLOCH FOUNDATION *(TO BE CREATED)* | POSTERN LABS LTDA *(EXISTS)* |
|---|---|---|
| Form | Non-profit; jurisdiction open | Brazilian limited company |
| Purpose | Steward the protocol; hold and distribute the non-founder allocations | Build the node, the OS, the apps, the wallet |
| Holds | 29% of supply at genesis, most of it vesting | — |
| Protocol authority | Publishes specs, GIPs, weak-subjectivity checkpoints | **None** |

Signing authority is split so the split means something: the Foundation signs listing agreements, VC subscriptions, grants, and checkpoints; Postern Labs employs the engineers and signs node releases — "whoever builds, signs" (§4 of the entity spec). The Foundation deliberately has no product revenue and Postern Labs deliberately has no protocol authority.

### 4.2 Who can change the rules

Three different kinds of power, kept distinct because conflating them is how governance descriptions mislead:

1. **Consensus rules** still change only the way they always did: candidate software, published, adopted or not by node operators. ADR-036 created no protocol lever; the supply cap has no setter (§2.5); there is no governance token and no on-chain voting on rules. Stake in the PoS design orders blocks and finalises checkpoints — it does not vote on rule changes. On this point Edition 1's Chapter 19 remains accurate.
2. **Real, named powers the Foundation holds** that are not consensus rules but shape the network: publishing weak-subjectivity checkpoints (under a phased m-of-n with a client-enforced external-signer minimum and a 12-month review, `BLOCH-WEAK-SUBJECTIVITY.md` §6 via the entity spec §5.3); administering 29% of supply including the delegation program; being the counterparty for listings and the VC round; publishing the genesis artifact. Each is a centralisation point and is named as one.
3. **Product decisions** belong to Postern Labs and bind nobody's node.

### 4.3 The honest clause

Until the independent board of `BLOCH-ENTITY-STRUCTURE.md` §5.2 exists — a majority of members unaffiliated with Postern Labs and with the founder, recusal of conflicted members from grant votes, published grant amounts and terms — **the two entities are the same people.** The spec says it itself, in the genesis-cohort context: "the Foundation is the founder until the board exists" (`BLOCH-TOKENOMICS-V4.md` §3.3.1). The Anza spin-out criticism that the entity spec opens with — "the same people, a new letterhead" — is the failure mode this structure reproduces by default, and the §5.2 controls are the only thing that would make it

otherwise. They are, at the time of writing, designed and not built: no jurisdiction, no board, no foundation.

What the structure does not fix, quoted from its own §7: a foundation makes concentration easier to **administer** — it does not make concentrated stake decentralised, and Foundation-delegated stake counted naively toward the gates is the specific way the structure could be used to appear to meet them without meeting them.

Open questions the spec routes to counsel rather than guessing: Foundation jurisdiction (with real Brazilian CFC/transfer-pricing exposure), who sells to the funds (Foundation directly or an SPV — the securities-sensitive act), board composition, and whether a third entity is ever warranted.

---

## 5. Validator bond and delegation

### 5.1 The bond: 25,000 BLCH

`crates/bloch-pos-committee/src/staking.rs`, `MIN_DEPOSIT_SAT` (founder decision, 2026-08-12; was 100,000 under the 21 B nominal). The figure is derived, not inherited: it is the fraction of supply Ethereum's 32-ETH bond represents, applied to the V4 supply and rounded **down** — exactly `supply / 4,000,000` — "on purpose: down is cheaper, and cheaper widens who *may* validate, which is the only direction a rounding choice on a bond should ever err" (the constant's own documentation). A pure ×100/21 split of the old floor would have landed at 19× the Ethereum-equivalent bond, so this constant deliberately does not follow the split; it is re-derived from the benchmark.

**What it solves:** the entry cost to *operating* a validator is pinned to the most widely accepted benchmark in staking, instead of an arbitrary number that would gate validation on wealth.

**What it does not solve**, stated in the file itself and repeated here because it is the sentence a promotional document would omit: "lowering the bond widens who MAY validate and does nothing about who DOES. It is not a fix for stake concentration and must not be described as one." The concentration facts of §3 are unmoved by any bond value.

### 5.2 Delegation

`crates/bloch-pos-committee/src/delegation.rs` — **built, not booted**, like everything else in the crate. Delegation exists because the adopted revenue model (Solana's: pro-rata inflation rewards scaled by attestation credits, commission on delegated stake, 50% base-fee burn during emission, priority fees to the producer — `BLOCH-TOKENOMICS-V4.md` §6.3) is meaningless without stake that can sit behind an operator without running one.

What makes it safe to add, each with its limit:

- **Rate-limited warm-up and cool-down** (`WARMUP_RATE_BPS`, floor `MIN_CHURN_SAT`, partial activation for oversized delegations) — instant activation would be instant control of stake-weighted committees. Limit: it slows entry, it does not vet it.
- **Delegated stake counts toward the 1% per-validator cap**, resolved by fixed-point iteration — delegation must not be a route around the cap. Limit: Sybil-splittable, as §3.3 states.

- **Delegators are exposed to slashing, pro-rata** — wired end-to-end (`transition.rs::apply_slashing_evidence` → `slashing.rs` → `delegation.rs::apply_slash`), with correlation amplification up to the entire delegated amount for coordinated equivocation. Otherwise delegation is all yield and no risk, and nobody cares who they delegate to. A delegator's principal is at risk for the operator's provable equivocation, not for mere downtime; there is no appeal path, because the evidence is two signatures the operator provably made.
- **Commission is uncapped and must be disclosed** — a cap is trivially evaded by an operator running its own delegation front-end, so the rule is disclosure (wallets and explorers must surface the rate), not limitation.

**What delegation solves:** it lets the validator set grow beyond the set of people willing to run infrastructure, it makes the Solana revenue model coherent, and — through the Foundation's delegation program — it solves the real bootstrap problem of a young validator set.

**What it does not solve, and can make worse if misread:** delegation makes the operator view *more* dispersed while beneficial ownership can remain exactly as concentrated as §3 measures. That is why Foundation-delegated stake is excluded from the gates by the §5.1 reporting rule, and why the gate metrics' own documentation concedes they "cannot see one beneficial owner standing behind several delegators, and no on-chain metric can."

---

## 6. Status ledger for this chapter

In the Edition 1 idiom, applied to everything above:

| ITEM | STATUS | MEANING HERE |
|---|---|---|
| ADR-036 retraction; ADR-033/034 retracted | Decided and recorded | A documentation and governance act — in force as a matter of record |
| Genesis-3 chain (PoW, SHA-256d, GhostDAG) | Booted | The live network today; scheduled to halt at the terminal height per `BLOCH-TOKENOMICS-V4.md` §3.2 |
| Tokenomics V4 constants, split, vesting functions, emission curves, cap invariant | Built, not booted | Standalone crate, not referenced by the node; active only if a Genesis-4 genesis is produced from it |
| Supply-cap enforcement (`SupplyCapExceeded`) | Built, not booted | Enforced by the PoS state transition, which no live chain runs |
| Delegation, per-validator cap, churn limit, slashing wiring | Built, not booted | Implemented and tested in `bloch-pos-committee`; no live validator set exists |
| Genesis validator cohort and declining cap | Built, not booted | `genesis_cohort.rs`; binds only a genesis that has not occurred |
| Bloch Foundation, board, jurisdiction | Designed | DRAFT spec; no legal entity exists |
| VC round | Designed | An allocation and vesting schedule in code; no counterparty exists yet to sign, no round closed |
| Weak-subjectivity checkpoint regime (m-of-n) | Designed | Parameters adopted on paper in `BLOCH-WEAK-SUBJECTIVITY.md` §6 |
| Phase-0 securities review | Not done — blocking | Reclassified from precautionary to blocking by ADR-036 |

| ITEM | STATUS | MEANING HERE |
|---|---|---|
| Third-party audit of the PoS crate | Not done | Pre-audit dossier exists (`docs/audit/CERTIK-PRE-AUDIT-DOSSIER.md`); gates any launch |

Nothing in this chapter is financial, legal, or investment advice; nothing in it is an offer; and no statement in it should be read as a claim about the present or future value of BLCH.

# Cryptography and security

# Bloch Institutional Dossier — Edition 2

## Cryptography & Security chapters (draft)

```
Document:   ED2-CRYPTO-SECURITY
Status:     DRAFT for Edition 2 assembly — supersedes Edition 1 chapters
            3 (The Post-Quantum Imperative), 10 (Cryptography),
            14 (ZK-Ledger Groundwork), 17 (Security Program),
            18 (Threat Model & Risk Factors)
Prepared:   2026-08-12, against branch integration/pos-modules
Repository: gitlab.com/blochsispow-group/bloch-pos (private at time of writing)
License:    AGPL-3.0-or-later
```

**Editorial contract, carried from Edition 1 and binding on every section below: designed ≠ built ≠ booted.** Having a specification, having code that implements it, and having that code running as the consensus-enforced rule on a live network are three different states, and every capability in this document is labeled with which of the three it has reached. One label matters more in Edition 2 than it did in Edition 1: **Genesis-4, the proof-of-stake chain these chapters describe, has not launched.** The consensus crate is substantially built and tested; the node around it is a devnet skeleton (localhost TCP, no transactions, no RPC — `crates/bloch-pos-node/src/engine.rs:95`, `net.rs:147`). Nothing in this document is "booted" unless it says so, and most of it does not.

---

## 1. The Post-Quantum Imperative

*This section preserves Edition 1's Chapter 3 nearly whole, because the argument has not changed and does not need to. What changed is the consensus mechanism underneath it — and, as Section 2 shows, proof of stake makes the argument bind harder, not softer.*

### 1.1 Harvest-now-decrypt-later

> **Harvest-now-decrypt-later** — an attack strategy in which an adversary records encrypted data, signed messages, or public keys today, with no capability to break them yet, in order to decrypt, forge, or otherwise exploit them once a sufficiently capable quantum computer becomes available.

The strategy is rational for any adversary who believes two things: that a cryptographically relevant quantum computer will exist within a time horizon shorter than the useful lifetime of the data or value being targeted, and that recording the target now is cheap relative to waiting. Both conditions plausibly hold for a public blockchain. Recording is nearly free — every public key and every signature on a public chain is, by definition, already public and archived by anyone who runs a node or an indexer. And the useful lifetime of value secured by a settlement layer is not bounded by a software release cycle; it is bounded by how long holders choose not to move their coins, which for a meaningful fraction of any chain's supply can be a decade or more.

This produces a threat model distinct from "is quantum computing a threat today." The correct question for a settlement layer is: by the time a cryptographically relevant quantum computer exists, will there still be unspent value secured by a signature scheme that computer can break? For Bitcoin and for essentially every classical elliptic-curve or RSA-secured chain, a meaningful fraction of supply sits in outputs whose public key has already been revealed on-chain — precisely the harvestable set. A protocol that has not made its signature scheme post-quantum before that computer exists faces a forced, urgent, contested migration under adversarial time pressure. Harvest-now-decrypt-later is not a hypothetical confined to some future chapter of cryptographic history; it is a present-tense collection activity against any classically-signed public chain today, whether or not a quantum computer capable of exploiting the harvest yet exists.

### 1.2 NIST PQC standards context

NIST's multi-year public standardization process produced the standards Bloch's cryptography is built directly on top of rather than around:

| STANDARD | ALGORITHM FAMILY | PRIMITIVE | ROLE IN BLOCH |
|---|---|---|---|
| FIPS 203 | ML-KEM (Kyber lineage) | Key encapsulation | ML-KEM-1024 where key encapsulation is required |
| FIPS 204 | ML-DSA (Dilithium lineage) | Signature, module-lattice hardness | ML-DSA-65, one half of the hybrid signature |
| FN-DSA track | Falcon | Signature, NTRU-lattice hardness | Falcon-1024, the second, independent half |

ML-KEM and ML-DSA derive hardness from structured module-lattice problems in the Kyber/Dilithium lineage; Falcon's hardness derives from NTRU lattices — a related but structurally distinct construction. That distinction is why the signature scheme does not simply pick ML-DSA and stop. The implementations are PQClean-derived, vendored, and frozen under `Cargo.lock`; any update to the vendored cryptographic lineage passes through a deliberate crypto-review gate rather than floating in with a routine dependency bump. Edition 2 must add one hard lesson to that sentence, learned since Edition 1: **a frozen lockfile pins versions, not feature flags** — see Section 2.

### 1.3 Why a hybrid AND construction

> **Hybrid AND construction** — a composite signature scheme in which a message is signed independently under two algorithms, and the composite signature is valid only if **both** component signatures verify. This is deliberately the conjunctive ("AND"), not the

> disjunctive ("OR"), combinator: an attacker must break both underlying hard problems simultaneously to forge a valid Bloch signature, not merely the weaker of the two.

Bloch requires ML-DSA-65 and Falcon-1024 to both verify on every spending authorization. This is live code, not a design note: the suite identifier `SUITE_MLDSA65_FALCON1024 = 0x0001` and its dispatch are in `crates/bloch-crypto/src/crypto/mod.rs:40` (sign at `:134`, verify at `:180–192`), with a documented, already-wired escape hatch `SUITE_MLDSA65_ONLY = 0x0002` (`:42`) that exists as the priced response to a failed review of the Falcon path — not as a pending consolidation. The three compounding reasons from Edition 1 stand unchanged:

1. **Algorithm diversity against an unknown future break.** A cryptanalytic advance against one lattice family is not automatically an attack on the other; a single-family break does not, by itself, forge a valid spend.
2. **Standardization-track redundancy.** FIPS 204 is final; Falcon sits on the FN-DSA track. Requiring both hedges against a future finding specific to either track.
3. **No implicit trust in a single implementation lineage.** A defect in one vendored implementation does not by itself compromise signature validity.

The cost is real and not hidden: two full post-quantum verifications per spend cost more compute and more bytes than either scheme alone. Edition 1's risk statement is retained verbatim in substance: the AND construction protects against a hard-problem break in one family; **it does not protect against an implementation bug that makes one verifier permissive.** Both codebases carry the same scrutiny burden as if each were the chain's only signature scheme.

The hashing layer follows the same logic at smaller scale: SHAKE-256 and SHA3-256 with explicit domain separation — distinct tags for key derivation, Merkle leaves, and Merkle internal nodes — so a hash computed in one structural role cannot be replayed in another. Every new consensus structure added since Edition 1 continues this discipline; Section 3's nullifier tree carries its own domain (`bloch:coherence:nfset:v1`, `crates/coherence-core/src/lib.rs:68`) precisely so its nodes can never be reinterpreted as commitment-tree nodes.

### 1.4 Migration risk

The trade stated in Edition 1 is restated without softening: Bloch chooses cryptography that is more resistant to a future quantum adversary and correspondingly less battle-tested against classical cryptanalysis than the incumbent schemes, on the judgment that harvest-now-decrypt-later makes that trade worth making now rather than later. Choosing post-quantum early does not remove risk; it changes which risk is carried. Nothing in this chapter is a claim that Bloch's cryptography is proven secure against all future attacks — the hybrid AND construction exists as a hedge against exactly that uncertainty, not as a claim that the uncertainty has been resolved.

---

## 2. What Changed in the Cryptography

The primitives did not change. ML-DSA-65 ‖ Falcon-1024 remains the signature scheme; SHAKE-256/SHA3-256 remain the hash layer; ML-KEM-1024 remains the KEM. What changed

is the *operating regime* of one of the two signers, and the discovery that the build system — not the source code — decided which implementation of it ran.

### 2.1 Proof of stake turns Falcon signing into a different problem

Under proof of work, a wallet signs occasionally, often from a machine that is offline the rest of the time, and an attacker does not know when. Under proof of stake, a validator signs **on a publicly known schedule, every assigned slot, indefinitely, from an internet-facing machine, with its bonded stake standing behind the key** (`docs/specs/BLOCH-FALCON-ONLINE-SIGNING.md` §1). A remote attacker knows in advance the exact wall-clock window in which a given validator will run the Falcon signing algorithm and can observe when the signed message appears on the network — which includes the signing latency. Over years of slots that is a high-quality repeated measurement of the same secret-dependent computation. Falcon is the floating-point-sensitive half of the hybrid: its Gaussian sampler is the reason NIST's FIPS 206 has lagged FIPS 204, and the reason "constant-time" is a per-implementation property, not a family property.

### 2.2 The finding: a transitive Cargo default selected native floating point

The adversarial review of the online-signing question (`docs/specs/BLOCH-FALCON-ONLINE-SIGNING.md` §3.2, finding F1) found that the requirement — integer-only, constant-time Falcon signing — was **violated by a build default, not by an availability gap**. The `pqcrypto-falcon` crate's default features (`["avx2", "neon", "std"]`) compile and runtime-dispatch Falcon signing to PQClean's native floating-point variants: AVX2 doubles on x86_64, `typedef double fpr` on aarch64. Every release binary on both server architectures was signing Falcon with hardware floating point — silently selected by a transitive default, on a machine that under PoS would sign on a public schedule with stake in bond. For the wallet-grade PoW threat model that configuration was defensible; for a scheduled online signer it is exactly the forbidden one.

### 2.3 The fix, and the two guards that keep it fixed

Falcon-1024 now signs exclusively through PQClean's `clean` variant — integer-emulated IEEE-754 binary64, written by the Falcon reference author, constant-time by construction, bit-exact across platforms. The pin and its guards, in decreasing order of how easily each could silently regress:

1. **The dependency pin.** `default-features = false, features = ["std"]` at both the workspace root (`Cargo.toml:133`) and the crate (`crates/bloch-crypto/Cargo.toml:60`).
2. **The dependency-graph guard.** Cargo features are additive: any workspace member or future dependency re-enabling the defaults would silently restore native-FP dispatch for every binary. `scripts/falcon-clean-guard.sh` fails CI if the workspace's *resolved* feature set for `pqcrypto-falcon` includes `avx2` or `neon` — and fails if the crate disappears from the graph, so a rename cannot turn the guard into a no-op (`scripts/falcon-clean-guard.sh:43-48`).
3. **The linked-binary tripwire.** The test `falcon_native_fp_variants_are_not_linked` (`crates/bloch-crypto/src/crypto/mod.rs:493`) inspects the compiled test executable itself for the native variants' symbols (`PQCLEAN_FALCON1024_AVX2_*`, `PQCLEAN_FALCON1024_AARCH64_*`), plus KATs pinning `clean` outputs so a dependency bump cannot silently re-enable dispatch (`crypto/mod.rs:400-406`, `:536-542`).

The measured cost of doing this correctly: 11.6 ms per Falcon-1024 signature on the integer path versus 0.46 ms native — ~25× slower and ~0.04% of a 30 s slot (`BLOCH-FALCON-ONLINE-SIGNING.md` §3.3). There is no engineering pressure to keep the floating-point path; the performance cost of correctness is a rounding error.

Verification is unaffected — Falcon verification is integer and deterministic on every path, so this was never a consensus change. Two protocol-level mitigations recommended alongside the pin — fixed-deadline publication (pad the one observable a remote attacker has, publication time, to a constant) and keygen-off-box (single-trace attacks published against Falcon target keygen) — are spec recommendations, not yet consensus rules (`BLOCH-FALCON-ONLINE-SIGNING.md` §5). Designed, not built.

### 2.4 A format boundary worth stating in an institutional document

Bloch's Falcon is and remains **Falcon-as-submitted** (the PQClean profile). The final FN-DSA standard will not be bit-compatible with it (NTT-form public keys, sampler changes). If the project ever wants FIPS-validated FN-DSA, that is a *new suite identifier* migrated through the existing envelope mechanism — a planned migration, never an in-place swap (`BLOCH-FALCON-ONLINE-SIGNING.md` §4). The envelope header (`crypto/mod.rs:37-58`) exists for exactly this class of event.

### 2.5 What did not change

The hybrid AND requirement did not weaken. The escape hatch to `SUITE_MLDSA65_ONLY` was priced and explicitly **not** pulled — its trigger is a P0 finding against the `clean` path under external review, not convenience (`BLOCH-FALCON-ONLINE-SIGNING.md` §6). And Edition 1's carryover note still applies in updated form: value created on the predecessor chain crosses into Genesis-4 as a snapshot of transparent balances (`docs/specs/BLOCH-TOKENOMICS-V4.md` §2), secured by the same hybrid key material; continuity of old value remains a stated obligation, not an accident.

---

## 3. Coherence — from "ZK-Ledger Groundwork" to a Frozen Format

Edition 1's Chapter 14 described groundwork: a deterministic execution substrate that happened to be the precondition succinct proofs need, an SP1 host-side stack, and a firm refusal to imply anything was live. Edition 2 can report that the groundwork acquired a name — **Coherence**, the shielded pool — and a frozen format, while keeping every one of Edition 1's honesty constraints intact.

### 3.1 What is now specified and built

The C1 specification froze the pool's formats: note structure, commitment and nullifier derivation, the commitment accumulator (SHAKE-256 Merkle tree, depth 32, domain-separated), the spend statement, and the wire format (`docs/specs/COHERENCE-C1.md`). The proof system is SP1's raw FRI-STARK — hash-based, **no trusted setup anywhere**, with the Groth16/Plonk wrapper paths explicitly forbidden (`crates/coherence-prover/README.md:44-53`). A hash-based proof system is also the choice consistent with this dossier's post-quantum posture: no pairing or discrete-log assumption is introduced through the proving back door.

### 3.2 C1.1 — the nullifier-set root (ratified 2026-08-12)

C1 named the global nullifier set as consensus state and never said how the set is committed; in code it was a bare `HashSet` with no canonical root. Under proof of work that was survivable, because nothing outside the node ever had to agree on the set's digest. Under proof of stake it is not: `state_root` commits every consensus component, so a state-syncing node must be able to check the set. C1.1 (`docs/specs/COHERENCE-C1.1.md`, the project's only RATIFIED Coherence spec) closes this:

- **The root is a sparse Merkle tree over the 256-bit nullifier keyspace**, SHAKE-256 throughout, keyed by the nullifier itself, under its own domain tag `DOM_NFSET = b"bloch:coherence:nfset:v1"` with depth 256 (`crates/coherence-core/src/lib.rs:68-92`). Empty subtrees short-circuit to precomputed roots, so cost is bounded by occupied paths, not the keyspace.
- **The root is a function of the set, not of history.** A running hash `H(prev ‖ nf)` was rejected because it makes insertion order consensus — two honest nodes applying the same blocks in different orders, or one redoing a reorg, would commit different roots for identical state (COHERENCE-C1.1.md §1.1). That failure shape caused a real incident on the predecessor chain, and reorgs are ordinary under PoS.
- **Non-membership is provable.** What a spend verifier needs is "this nullifier is not in the set as of this anchor," and a sparse tree can prove it where a hash chain cannot: `NullifierSet::non_membership_proof` and `verify_non_membership` (`crates/coherence-core/src/lib.rs:123`, `:243`), pinned by tests including the adversarial direction — a spent key must not verify as absent (`lib.rs:599-702`).
- **One implementation.** `coherence-core` computes the root; the node, the SP1 guest, and the genesis ceremony all call the same code. The PoS consensus crate carries the 32-byte root as opaque bytes and never recomputes it — the interim commitment the PoS layer had been computing on the pool's behalf was removed by this rev (COHERENCE-C1.1.md §1.3).
- `insert` returning `false` on a present nullifier **is the double-spend check**; `remove` exists solely for reorg undo and restores the exact earlier root, pinned by test (COHERENCE-C1.1.md §1.2).

### 3.3 What Edition 1 refused to claim, Edition 2 still refuses to claim

Carried forward deliberately, in the same words the ratified spec uses (COHERENCE-C1.1.md §3):

- **No privacy claim is made** until the planned external audit of the Coherence stack (C4). The chain remains, in this respect, a zero-security testnet.
- **The value bridge does not exist.** `shield_tx`/unshield is undefined; value cannot enter or leave the pool. A shielded pool with no inlet is a format and an accumulator, not a privacy feature.
- Shielded-transaction application under PoS is not implemented; the pool is inert — headers carry forward the parent's committed roots and no block changes them yet. There is no shielded storage; a restarted node rebuilds an empty pool.

Designed: the C1/C1.1 formats, frozen and ratified. Built: `coherence-core` (708 source lines, 12 tests, its only dependency `sha3`), the prover host-side stack, the genesis-ceremony carry logic. Booted: nothing — no live proof pipeline, no consensus-wired verifier, no committed

timeline. Edition 1's closing caution applies unchanged: this is a description of direction and precondition, not of a shipped feature.

---

## 4. The Security Program — and the Findings We Found Ourselves

Edition 1's security chapter named the instruments (cargo-audit, cargo-deny, cargo-geiger, hardened Clippy, Miri, cargo-fuzz, proptest, OSS-Fuzz, a blocking supply-chain CI gate) and disclosed what none of them can certify. All of that stands. What Edition 2 adds is the strongest evidence a security program can offer: **the defects it found in its own consensus code, before any auditor did, each fixed with a commit and pinned with a regression test.** The full record, with reproduction detail, is `docs/audit/CERTIK-PRE-AUDIT-DOSSIER.md` §3; this section is the institutional summary. An auditor reads a list like this as maturity. Hiding it would read as risk.

### 4.1 Seven findings, seven fixes, seven regression tests

**F-1 — Two block identities.** At integration the interfaces crate had a public tuple `BlockId` mintable from any 32 bytes coexisting with the opaque header identity; two disagreeing header types and three copies of the canonical serialization existed — nine construction sites where the design allows one. Fixed in `62ca5af`. The guard is structural, not a review note: `single_derivation_path` (`crates/bloch-pos-committee/src/header.rs:626`) scans the crate source and fails the build on any second construction site, any manual trait impl, any alias — because this exact class of bug (a DAG keyed by one hash while storage keyed by another) caused a real outage on the predecessor chain.

**F-2 — The header did not commit to the body, in the code the node actually runs.** `compute_post_state` never checked `body_root`, `attestation_root`, or `coherence_root`; those checks lived only in a parallel validator with no caller, and 178 green tests said nothing because the test builder stamped zeros. One `block_id` could name two different bodies. Fixed in `d29e3ad`: the transition recomputes all three roots. Regression: `header_must_commit_to_body_attestations_and_coherence` (`src/transition.rs:2753`), asserting all three mismatch errors in both directions.

**F-3 — Two state-root derivations.** `transition` and `derive` committed *different* RANDAO leaf sets for the same block — two roots for one block, each seam green on its own tests; found independently by two reviewers the same day. Fixed in `4ca2646`: one function, `state_root::randao_window` (`src/state_root.rs:924`); the stricter rule won. Regression: the five tests of `tests/one_state_root.rs` — reverting the old rule fails two of five.

**F-4 — Slashing existed but nothing called it; then its bookkeeping was not under the root.** `slashing.rs` was implemented, tested, sealed — and had no call site. Fixed in `8a3e0ea`: evidence became a transaction validated inside the state transition; invalid evidence rejects the whole block; forged evidence slashes nobody. Then the anti-replay set, correlation window, and delegator-loss ledger were found uncommitted under `state_root` — a state-synced node could double-slash. Fixed in `319c7e6` with three new committed leaves (`src/state_root.rs:148-150`). Regressions: `src/transition.rs:2198`, `:2372`, `:2518` (`every_committed_state_field_is_bound_by_the_root`), `:2815`.

**F-5 — The Ustav supply cap was not a cap (HIGH).** `compile_supply` compared the cap against a value the *spender* wrote in the redeemer, never against the amount actually minted: with `cap = 1,000` an issuer minted 1,000,000,000 in one transaction. Since the program hash is the asset's policy id, promoting Ustav to consensus would have made the chain certify a false cap. Fixed in `0f67977`: the module reads the mint context and asserts `prior + delta <= cap`. Regressions: `supply_cap_cannot_be_bypassed_by_a_redeemer_supplied_amount` (`crates/bloch-euvm/src/modules.rs:725`) and `tests/audit_modules_supply.rs:55`, `:135`.

**F-6 — Redeemer padding bypass defeated the freeze control (CRITICAL).** The VM imposed no redeemer arity; compiled modules read the `frozen` flag at a fixed top-relative stack offset, so padding the redeemer with one extra value shifted every read onto attacker-controlled data — a frozen regulated-token output became spendable with no authority signature. Fixed in the same `0f67977`: a new `Op::ExpectDepth` opcode (`crates/bloch-euvm/src/lib.rs:136`, executed at `:379–383`) is emitted as the first op of every compiled module, making expected arity part of the program and hence of `validator_hash` — the spender cannot renegotiate it. Regression: `tests/audit_modules.rs:55`, which deliberately keeps its bug-era name with the assertion inverted.

**F-7 — The shielded pool did not cross the genesis seam, and its nullifier set had no canonical root.** The Genesis-3→4 carryover pipeline moved only transparent balances; the ceremony stamped `coherence_root = [0u8; 32]` ("empty pool") with nothing carrying the tree or the nullifier set — a pool that does not cross as ordered leaves plus the complete nullifier set either burns every unspent note or revives every spent one. Worse, the header's coherence mirror was copied from the parent verbatim, validating nothing. Fixed in `eacddd9` (merged `c59a175`): the ceremony requires a fail-closed Coherence artifact, replays leaves in exact position order, carries the nullifier set whole, and "empty" became an attested artifact, never an assumption. Follow-up `c6fe0c1` ratified C1.1 (Section 3.2 above). Regressions: `src/derive.rs:699`, the ceremony tests at `tools/genesis4-ceremony/src/lib.rs:1675–1735` (including `dropping_a_nullifier_is_a_different_chain`), and the seven C1.1 set tests.

### 4.2 The pattern, named

F-2, F-3, and the second half of F-4 are one class: **a consensus value with two derivation paths, or none.** The codebase now carries structural tests against the class itself, not just the instances — `single_derivation_path`, `tests/one_state_root.rs`, and `every_committed_state_field_is_bound_by_the_root`. That is the security program's actual output: not a clean bill of health, but a machine that converts each found defect into a permanent guard against its class.

### 4.3 What the program still cannot certify

Unchanged from Edition 1, and restated because it is the most important paragraph in the chapter: internal adversarial review and automated scanners reduce risk measurably — the findings above are real defects that would otherwise have shipped — but they are not a substitute for independent third-party review. Consensus-logic correctness under adversarial network conditions, the soundness of the specific cryptographic composition, bridge-class logic, and economic security all sit outside what static analysis, fuzzing, and property testing are built to catch. Section 6 states where third-party review stands.

---

## 5. Threat Model, Updated for Proof of Stake

Edition 1's threat model led with 51%-attackability at low hashrate. That risk died with the PoW chain; the risks that replaced it are not smaller, and several have no PoW analogue at all. The primary sources are the two adversarial passes `docs/specs/BLOCH-POS-THREAT-MODEL.md` (F1–F13) and `BLOCH-POS-THREAT-MODEL-2.md` (G1–G4), kept deliberately unrewritten as a record, with status seals recording what each finding's fate was. The findings below are stated with their current status; a finding marked *closed* means closed by wired, tested code, because the second pass's headline lesson (G1) was precisely that **a correction that is not wired is not a correction**.

### 5.1 Findings from the adversarial passes, and where they stand

- **F1 (Critical) — the finality quorum had no coherent denominator.** The epoch committee was a 128-validator sample; ⅔ of network stake was unreachable (permanent finality stall) and ⅔ of committee stake let a sub-⅓ adversary stall via sampling variance. Closed by replacing the sample with a **partition of the whole active set** — and G1 caught that the partition first landed as dead code with no caller while the seal claimed the fix. It is now wired at `finality::votes_from_partition` (`crates/bloch-pos-committee/src/finality.rs:768`).
- **F2 (High) — honest validators would self-slash**, because per-slot and epoch attestations shared one struct and one signing domain, so routine multi-slot duty satisfied the `DoubleVote` predicate against itself. Closed alongside F1 by the partition and role separation.
- **F6 (Medium) — beacon grinding of the next epoch's committee.** Trailing- slot proposers could withhold reveals to grind the mix that seeds the next epoch's committees. Closed by a seed look-ahead (`MIN_SEED_LOOKAHEAD_EPOCHS`, `src/committees.rs:51-88`): epoch N is seeded by the mix fixed at the close of N–2, with the honest residue stated — the one-bit-per-withheld-slot bias is displaced an epoch forward, not eliminated (the same residue Ethereum accepts).
- **F7 (Medium) — accepted surface:** public sortition hands an attacker the per-slot committee schedule one epoch ahead, making targeted DoS cheap. Quantified in `BLOCH-POS-SORTITION-DOS.md`; mitigations (sentry nodes, late attestation inclusion, proposer boost) are recommendations, not yet rules. Accepted and disclosed, not closed.
- **F8 / stake-churn — decided and applied:** the warm-up rate was cut from 900 to 25 bps/epoch with a churn floor (`WARMUP_RATE_BPS = 25`, `src/delegation.rs`), turning a zero-to-⅓ stake shift from ~75 minutes into ~43 hours — visibility time, not prevention. The per-validator 1% cap remains honest about its limit: **Sybil-bypassable by splitting**, per its own doc comment. No on-chain rule sees beneficial ownership.
- **G2 (High) — the decentralization rule could halt the chain.** The genesis-cohort cap, engaging ~1.3 hours after genesis, would zero the entire active set's effective stake if no independent stake had arrived — the rule meant to decentralize the founder would stop the network. Closed by deferring the cap until independent stake exists (`CapStatus::Deferred`, `src/genesis_cohort.rs:100-124`) — with the honest cost that the taper is now gated on adoption, and the known drift that this consensus behaviour is in code but absent from the spec (`BLOCH-POS-GAPS.md`, via CERTIK dossier §4 item 10).

- **F10 — weak subjectivity is an operational dependency, not a window.** The 2048-epoch withdrawal delay (22.76 days) is the maximum age of a usable checkpoint: a node syncing from anything older can be fooled by a long-range fork built with since-withdrawn keys. Fresh checkpoints must be published on cadence, and that duty sits with the Foundation — a standing operational dependency, stated as such.

### 5.2 Risks proof of stake creates that proof of work did not have

1. **The signer is online, on a schedule, with value bonded behind the key.** The entire Section 2 story. Under PoW, consensus keys did not exist and wallet keys could live cold; under PoS the validator key is a hot key whose compromise is slashable stake, and whose side-channel surface is measured in signatures per year (~tens of thousands).
2. **Slashing is a new way to destroy honest value.** A logic error in the slashing predicate (F2) or an unauthenticated evidence path punishes honest validators. The mitigations are in code — invalid evidence rejects the whole block, forged evidence slashes nobody, both regression-tested (Section 4.1, F-4) — but the category itself is new: PoW had no protocol mechanism that could confiscate a miner's capital.
3. **Long-range forks and weak subjectivity.** PoW finality degraded gracefully with hashrate; PoS finality is stronger inside the subjectivity horizon and undefined outside it without a trusted checkpoint (F10). Syncing from genesis alone is no longer sufficient.
4. **Randomness is now consensus.** PoW needed no beacon; PoS committee selection does, and the beacon is grindable at the margin (F6/G3 — bounded and displaced by the look-ahead, not eliminated).
5. **The schedule is public.** Targeted DoS against the exact machines that carry a slot's fork-choice weight is a PoS-specific, pre-announced target list (F7).
6. **Stake concentration is consensus power directly.** Under PoW the founder's balance could not vote; hashrate was external and had to be bought. Under PoS the founder's carried-over balance is stakeable by decision of record, and if staked while others abstain it is ~94% of active stake — **a Nakamoto coefficient of 1** (`docs/audit/CERTIK-PRE-AUDIT-DOSSIER.md` §1.1, measured, not estimated). The consensus-coded bounds (genesis-cohort taper, per-validator cap, churn rate) constrain the founder's *validator weight*; they do not and cannot constrain the founder's *holdings*. Any framing of those mechanisms as fixing concentration would be false, and this dossier does not offer one.
7. **Bootstrap liveness.** A fresh PoS genesis must produce block 1 with whatever stake exists at slot 0 — the class of self-inflicted halt G2 found. PoW chains bootstrap on any nonzero hashrate; PoS chains need a staked, attesting set from the first epoch.
8. **Key ceremonies that do not exist yet.** No production Genesis-4 key exists; the custody plan is DRAFT (`docs/specs/BLOCH-GENESIS-KEYS.md`). The Genesis-3 keys guarding the liquid carryover were generated long ago under unknown conditions and sit outside that plan, inside the risk.

### 5.3 What was retired with proof of work

Stated for completeness, because a threat model that only adds is not being honest about the trade: rented-hashrate 51% attacks on a low-hashrate SHA-256d chain — Edition 1's leading risk — no longer apply. The successor question, "what does it cost to acquire a stake position that threatens consensus," currently has a degenerate answer: nothing needs acquiring,

because the founder already holds it (item 6 above). Decentralization is measured by the G1–G11 gates, and **no gate has an observed value**, because the chain has not launched.

---

## 6. Audit Status, Stated Honestly

**No independent third-party audit of this codebase has been completed.** `docs/SECURITY_SELF_ASSESSMENT.md` records zero external audits to date. The prior internal audits (`docs/audit/AUDIT-2026-04-20_ERA1.md`, `docs/audit/groundstate_audit.md`) cover a predecessor codebase. What exists today:

- **A pre-audit dossier prepared for engagement** (`docs/audit/CERTIK-PRE-AUDIT-DOSSIER.md`), which answers a token-scanner's checklist property-by-property for an L1, defines the real audit surface (four crates, ~37k source lines, 814 passing tests, measured not asserted), hands over the self-found findings of Section 4 with commits and regression tests, and lists the open gaps it would not want an auditor to discover first — including that the PoS node is a devnet skeleton, that genesis keys do not exist, and that the repository itself is private at time of writing.
- **A defined external-review requirement before launch:** gate G7 requires external review of exactly the highest-value target — the online Falcon-1024 signing path of Section 2 (`docs/specs/BLOCH-FALCON-ONLINE-SIGNING.md`; CERTIK dossier §2).
- **Two applicable checks that fail today and are written as FAIL:** holder concentration (Nakamoto coefficient 1 if the founder stakes) and open-source publication (the repo is private, and the decentralization ADR rests on publication before launch). No reframing is offered.

The engagement, when it happens, would be the first external audit. Until it concludes: the hybrid signature composition is unreviewed by third parties, the Coherence stack makes no privacy claim (Section 3.3), activation of the contract VM remains gated on an audit that has not occurred, and anyone relying on this protocol for value at risk should weigh those gaps accordingly. Designed ≠ built ≠ booted — and separately from all three: **unaudited**.

# Appendix — pre-audit dossier

## Bloch — CertiK pre-audit dossier

Document: CERTIK-PRE-AUDIT-DOSSIER Status: DRAFT for founder review — prepared before engagement, to be handed to CertiK Prepared: 2026-08-12, against branch `integration/pos-modules` at `470b608` Supersedes: the 2026-08-12 draft prepared at `84ca42a`, which predates the 100-billion redenomination, the supply-cap consensus invariant and the halt height moving to 39,918 Repository: `gitlab.com/blochsispow-group/bloch-pos` (private at time of writing — see check 20) License: AGPL-3.0-or-later (repo `LICENSE`; workspace `Cargo.toml:24`)

---

### 0. What this document is, and one thing it refuses to do

CertiK's Skynet **token scan** (the model given to us: the WBNB scan on BSC, 22 passed / 1 attention / 0 alerts) is a bytecode analyser for deployed EVM contracts. It reads a contract at an address and looks for mint functions, owner modifiers, proxy slots, tax logic.

**That scanner cannot be run against Bloch, and this dossier will not pretend otherwise.** BLCH is the base asset of an L1, not a contract: there is no address to scan, no owner to renounce, no proxy slot to inspect. EVM at L1 is a design in progress (`docs/specs/BLOCH-L1-EVM-STATE-MODEL.md`), not a deployed token, and there is no Skynet listing to apply for until that surface exists.

What we can do — and what this dossier does — is answer, for every check the scanner runs, whether the *property behind the check* holds on Bloch, by what mechanism, and where the evidence is. A check that does not apply is never answered with a bare "N/A": it gets "does not apply because X, and the thing that plays its role here is Y" — and where nothing plays that role, the verdict is FAIL and is written as FAIL.

What CertiK would actually audit for a chain like Bloch is the **node and consensus code**, not a contract. Section 2 defines that surface, with measured line counts and measured test results. Section 3 lists the findings we found and fixed ourselves, with commits and regression tests. Section 4 lists the open gaps without softening. Section 5 flags internal contradictions we found while preparing this document, because an auditor would find them anyway.

Verdicts used below:

- **PASS** — the property holds, evidenced in code.
- **FAIL** — the property does not hold. No reframing.
- **N/A — substitute** — the check is meaningless for an L1 base asset; the row names the mechanism that plays its role and whether *that* holds.

---

## 0.1 Decisions of record — what changed on 2026-08-12

An auditor reading an earlier copy of this dossier will find different numbers. These are the changes, and the reason each is stated here rather than quietly folded in is that a document whose figures move without a record is worth nothing to an auditor.

| DECISION | BEFORE | NOW | WHERE IT LIVES |
|---|---|---|---|
| Total supply | 21,000,000,000 (superseded) | **100,000,000,000** | `tokenomics_v4.rs` |
| Method | — | Pure redenomination, ratio `100/21` applied to every allocation. Shares unchanged, nobody diluted. Compile-time assertions prove each bucket scaled by the same ratio. | `tokenomics_v4.rs` (`SPLIT_NUMERATOR` / `SPLIT_DENOMINATOR`) |
| Supply cap | Constant respected by construction | **Consensus invariant.** Cumulative issued supply is a committed state component; a block whose pre-state exceeds the cap is rejected. | `state_root.rs` tag `0x14`; `transition.rs` (`SupplyCapExceeded`) |

| DECISION | BEFORE | NOW | WHERE IT LIVES |
|---|---|---|---|
| Validator bond | 100,000 BLCH (superseded) | **25,000 BLCH** — Ethereum's 32 ETH as the same fraction of supply, rounded down | `staking.rs` |
| Genesis-3 halt | height 80,000 | **height 39,918** | `crates/bloch-crypto/src/core/mod.rs` |

Three consequences that are costs, not details, and that an auditor should see named rather than discover:

1. **10^19 satoshis does not fit a signed 64-bit integer.** The supply is 54.2% of `u64::MAX` and 108% of `i64::MAX`. Every quantity inside the consensus crate is `u128` and unaffected. The Go SDK types `Satoshis` as `int64` and **must migrate before Genesis-4 ships**; so must any exchange integration that made the same choice. The compile-time assertion was inverted to state this rather than deleted.
2. **The emission curve had to be re-derived, not scaled.** It is denominated in absolute satoshis, so leaving it unchanged would have cut year-one inflation from 4.37% to 0.91% — a monetary policy change wearing the costume of a unit change. Multiplying it by `100/21` instead *overshot* the allocation by 65,042,160 sat, because integer truncation inside the per-slot division does not commute with scaling. The constant is now the largest value whose 40-year sum stays under the allocation, found by binary search over the emitter's own recurrence, leaving 176,880 sat unissued.
3. **Per-address conversion cannot be exact.** `100/21` never divides a power of ten, so no choice of decimal places makes individual balances scale exactly. Conversion is floor division; the remainder is absorbed by the founder's carried-over balance so the total closes at the cap. Bucket totals *are* exact — every allocation is already a multiple of 21 million.

---

## 1. The token-scan checklist, answered for an L1

The WBNB model scan evaluates the 23 checks below (scored 22 passed, 1 attention — the attention being major-holder ratio at 39.28%). File references are to this repository at `470b608`.

| # | CHECK | APPLIES TO BLOCH? | VERDICT | EVIDENCE |
|---|---|---|---|---|
| **Market** | | | | |
| 1 | Buy tax | No — no contract, no tax logic on acquiring BLCH. Role played by: base-layer transaction fee under an EIP-1559-style controller; no party receives a privileged cut (the burned share is burned by never being credited). | N/A — substitute holds | `crates/bloch-pos-committe` ( `next_base_fee` ); `src/rewa` ( `split_fees_at` ); `src/trans` |
| 2 | Sell tax | No — same as check 1; disposing of BLCH pays the same fee as any transaction, to no privileged party. | N/A — substitute holds | Same as check 1 |

| # | CHECK | APPLIES TO BLOCH? | VERDICT | EVIDENCE |
|---|---|---|---|---|
| 3 | Buy restrictions | No contract gate. Role played by: the closed transaction set — `Transfer` is one of exactly five `PosTransaction` variants, with no admission oracle; the provenance/taint machinery is retired-inert by consensus ("no eligibility oracle may produce `true`"). | N/A — substitute holds | `crates/bloch-pos-committe` `220` ; `src/staking.rs:172-18` |
| 4 | Sell restrictions | No contract gate. Role played by: permissionless spend/exit/withdrawal. The only delays are stake-scoped and disclosed (`WITHDRAWAL_DELAY_EPOCHS`, delegation cool-down); withdrawal validation is a pure predicate with no approver. | N/A — substitute holds | `crates/bloch-pos-committe` `:489-506` ; `src/delegation.` |
| 5 | Anti-whale mechanism | No transfer/balance limits exist (in token-scan terms, none of the risk-flagged kind). Stake-side concentration bounds exist: per-validator cap of 1% of active stake, genesis-cohort declining cap, 25 bps churn. All three are honest about their limits: Sybil-bypassable by splitting, per their own doc comments. See §1.1. | N/A — substitute exists, with stated limits | `crates/bloch-pos-committe` `:103` ; `src/genesis_cohort.` `src/transition.rs:590` , `:8` |
| 6 | Anti-whale modifiability | No privileged role can modify the caps in check 5: they are compile-time constants with no setter, no governance hook, no admin path (see check-23 grep). Changing them requires a code change every operator must adopt (a hard fork). | PASS (via substitute) | `crates/bloch-pos-committe` `110` ; §1.3 grep table |
| 7 | Honeypot (can buy, cannot sell) | No code path can block disposal: exit is self-signed, withdrawal needs no signature at all and has exactly three structural reject reasons (`NotExited`, `DelayNotElapsed`, `AlreadyWithdrawn`). Honest caveat: disposal is *market*-limited (no exchange listing, thin liquidity) — a fact for the listing conversation, not a protocol mechanism. | N/A — substitute holds | `crates/bloch-pos-committe` `:470-479` , `:489-506` |
| 8 | Self-destruct | No `selfdestruct` analogue, and no hidden kill switch (check-23 grep). One disclosed, deliberate exception must be named: Genesis-3 carries a **terminal-height consensus rule** — the chain halts at a fixed height for the Genesis-4 migration. It is public, one-time, and snapshot-preserving, not a concealed destruct path. | N/A — substitute holds, with disclosure | `docs/specs/BLOCH-TOKENOMI` `BRIEF-2026-08-11.md` |

| # | CHECK | APPLIES TO BLOCH? | VERDICT | EVIDENCE |
|---|---|---|---|---|
| | | See §5 for the height contradiction. | | |
| 9 | **Major holder concentration** | **Applies directly — the one check that maps one-to-one.** Measured: the largest single address holds **93.94%** of the carried-over supply (17,046,829,380 of 18,146,400,000 BLCH, snapshot at height 39,918, 16 addresses); **70.5%** of circulating at slot 0; if staked (it is stakeable, founder decision 2026-08-11), **94.0% of active stake — Nakamoto coefficient 1**. WBNB drew its only attention flag at 39.28%. | **FAIL** | `docs/specs/BLOCH-TOKENOMI` `280d604b32525f03…` ), §4A, § `committee/src/tokenomics_v` ( `LARGEST_CARRYOVER_ADDRES` §1.1 |
| 10 | Mintable | No mint function, no privileged issuance: `PosTransaction` is a closed five-variant enum with no `Mint` ; the only balance-increasing writes in the transition are the emission curve and fee compounding, both pure functions of slot/stake. Open gap: the decided block-level cap invariant is **not yet implemented** — see §1.2. | PASS (no privileged mint) — with an open gap on the cap invariant | `crates/bloch-pos-committe` `220` , `:1152` , `:1177` , `:1186-` `src/tokenomics_v4.rs:406-` |
| 11 | Blacklist | None. A crate-wide grep for blacklist/freeze/ban/censor machinery returns prose and retired-inert fields only (§1.3). The Genesis-3-era taint set is dismantled by named-zero constants so anything still consulting it fails loudly. (Token-level `Gate::Deny` exists in `bloch-euvm` — a regulated-asset primitive in a crate that is **not consensus-wired**; see §2.) | PASS | §1.3 grep table; `crates/bloc` `committee/src/staking.rs:` `src/tokenomics_v4.rs:106` ; `euvm/src/state.rs:558-575` |
| 12 | Whitelist | None at chain level — no allowlist gates participation in transfer, staking, delegation, or block production beyond the public parameter thresholds. (`MembershipList` in `bloch-euvm` is token-scoped and not consensus-wired.) | PASS | §1.3 grep table; `crates/bloc` |
| 13 | Hidden ownership | No hidden control mechanism (§1.3 grep). Governance is explicitly **not** ownerless — the earlier "ownerless" claim was formally retracted — and the structure is disclosed: two entities, founder allocates the genesis validator cohort under a consensus-coded taper. Honest limit: beneficial ownership of the 14 non-founder carryover addresses is asserted, not | PASS (disclosed, not hidden) — with the attribution caveat | `docs/adr/ADR-036-*` ; `docs/` `STRUCTURE.md` ; `crates/bloc` `committee/src/genesis_coh` |

| # | CHECK | APPLIES TO BLOCH? | VERDICT | EVIDENCE |
|---|---|---|---|---|
| | | provable on-chain — the tokenomics doc treats the whole non-founder remainder as "independent parties" without attribution. | | |
| 14 | Proxy contract (upgradeable logic) | No proxy slot exists. Role played by: node releases — operators choose what to run, and the release-integrity discipline (reproducible builds, published == fleet binary) is the substitute control. That discipline is specified (gate G8, `REPRO.md`) but **G8 is unmeasured** (§4). | N/A — substitute specified, not yet measured | `REPRO.md`; `repro-manifest.` `SHA3-LATTICE-MIGRATION.md` |
| 15 | Balance modification (privileged) | No privileged balance write exists. Slashing is the only involuntary balance reduction and it requires structurally valid evidence validated inside the state transition — invalid evidence rejects the whole block; forged evidence slashes nobody (regression-tested). | PASS | `crates/bloch-pos-committe` `:1186-1188`, `:1482-1489`; `s` `:2198`, `:2372` |
| 16 | Tax modification by privileged roles | The fee split constants have no setter and no role that can change them; the base fee moves only by the in-protocol controller as a function of block usage. Change requires a hard fork. | PASS (via substitute) | `crates/bloch-pos-committe` `:65-75`; `src/fee_market.rs` |
| 17 | Transfer cooldown | None on transfers. The delays that exist are stake-scoped, constant, disclosed, and apply equally to everyone — including the founder (`ACTIVATION_DELAY_EPOCHS`, `EXIT_DELAY_EPOCHS`, `WITHDRAWAL_DELAY_EPOCHS`, delegation `COOLDOWN_EPOCHS`). | PASS | `crates/bloch-pos-committe` `:106`; `src/delegation.rs:1` |
| 18 | Transfer pausability | No pause authority. The only halt-shaped code is (a) the node-local `HaltForOperator` on a checkpoint conflict, which by test **cannot** override a node's own finality, and (b) the disclosed one-time terminal height (check 8). | PASS — with the terminal-height disclosure | `crates/bloch-pos-committe` `:1112` (`published_checkpoint_nev` |

| # | CHECK | APPLIES TO BLOCH? | VERDICT | EVIDENCE |
|---|---|---|---|---|
| 19 | Ownership renunciation | Does not apply — no owner object to renounce. Role played by: the genesis-cohort consensus rule that tapers the founder-operated cohort's combined weight from 100% to below one third within one year (a shrink-only, publish-once set that is part of chain identity). Honest limit: unlike a renounced contract, the founder retains real control today — this row does not offset check 9's FAIL. | N/A — substitute exists, control today remains with the founder | `crates/bloch-pos-committee/src/genesis_coho` `src/transition.rs:2647–26` `src/transition.rs:2408` |
| 20 | **Open-source code** | Applies directly. Licensed AGPL-3.0-or-later throughout, and the decentralisation ADR rests on "code open-sourced *before* launch" — but **the repository is private at time of writing**, and no document commits to a publication date, URL, or process. The predecessor Genesis-3 repo and binaries are public; this one is not yet. | **FAIL** (today) — pass requires publication before engagement or a committed date | `LICENSE:1` ; `Cargo.toml:24` ; `license-pos-crates.md` ; `do` `decentralization-model.md:` |
| 21 | External calls | No consensus-time external calls: no oracles, no cross-contract calls, no network I/O inside validation. Role played by: dependency supply chain, governed by `cargo-deny` / `cargo-audit` config and a vendored, symbol-pinned PQ stack; the Coherence prover (SP1) runs client-side — nodes verify proofs, never call out. | N/A — substitute holds | `deny.toml` ; `audit.toml` ; `cr` `crypto/src/crypto/mod.rs:4` `crates/coherence-prover/R` |
| 22 | Withdrawal function (owner can drain) | No contract balance and no function that drains one. Role played by: key custody for the genesis allocations — the Foundation custodies 29% of supply at genesis, and the custody plan (air-gap ceremony, sharding, exposure windows) is written but DRAFT, and **no production key exists yet** (§4). Institutional custody, not protocol code, is the real surface here. | N/A — substitute is a DRAFT plan, flagged as a gap | `docs/specs/BLOCH-GENESIS-` `docs/specs/BLOCH-ENTITY-S` `docs/research/MOFN-CUSTOD` |
| 23 | Backdoor ownership recovery | None. The crate-wide grep for owner/admin/sudo/master-key/override/emergency paths (§1.3) returns no control mechanism — every hit is prose, a frozen-interface artifact, or the retired taint set. | PASS | §1.3 grep table |

Score, stated plainly: of the two checks that apply one-to-one, **both fail today** — concentration (9) and open-source publication (20). Of the remainder, the substitute mechanisms hold in

code, with three carrying open caveats (10: cap invariant unimplemented; 14: G8 unmeasured; 22: custody plan DRAFT).

### 1.1 Concentration — the exact numbers, and what bounds them (check 9)

This is the finding an auditor will lead with, so we lead with it.

Measured, not estimated (snapshot at Genesis-3 height 39,918 via `bloch-snapshot-utxo`, snapshot root SHAKE-256 `280d604b32525f03…`, carryover digest `92918209a106f297…` — `BLOCH-TOKENOMICS-V4.md` §2):

- The carryover set is 18,146,400,000 BLCH across **16 addresses** and 452,726 UTXOs. The largest address holds 17,046,829,380 BLCH — **93.94% of the carryover** — and is the founder's.
- At Genesis-4 slot 0, circulating supply is carryover + liquidity + marketing TGE; the founder's liquid balance is **70.5% of circulating**.
- The carried-over balance is liquid and therefore **stakeable** (founder decision 2026-08-11, pinned by `staking.rs::carryover_liquid_balance_is_stakeable` and `tests/committee.rs::carryover_liquid_balance_delegates_as_stake`). If it stakes and others do not, the founder holds **94.0% of active stake — a Nakamoto coefficient of 1**, computed at the one-third threshold.
- Founder total across carryover plus the new 10% grant (10-year cliff, 40-year linear vest): **27.04% of eventual total supply**.

The mechanisms that bound this, and — stated with equal weight — what each does *not* reach:

| MECHANISM | WHAT IT DOES | WHAT IT DOES NOT DO |
|---|---|---|
| Genesis-cohort declining cap (`genesis_cohort.rs:75–81`, `:129–194`; enforced at `transition.rs:590`) | Consensus rule: the founder-operated genesis cohort's combined duty weight tapers 100% → 33.3% over 12 months; the cohort list is publish-once, shrink-only, part of chain identity. | Binds only the named cohort. Nothing stops funding new validators at addresses outside it; no on-chain rule sees beneficial ownership. |
| Per-validator cap: 1% of active stake (`delegation.rs:103`; fixed-point at `delegation.rs:337–349`; `transition.rs:857–858`) | No single validator identity exceeds 1% of effective stake. | Sybil-bypassable by splitting one balance across many validators — the code's own doc comment says so. |
| Churn rate 25 bps/epoch (`delegation.rs:90`, was 900; `BLOCH-POS-STAKE-CHURN.md`, ACCEPTED AND APPLIED) | Slows stake movement: a stalling third now takes ~43 hours instead of ~75 minutes — visibility time for operators. | Time, not prevention. Does not reduce anyone's holdings. |

And the arithmetic that closes the escape hatch (`BLOCH-TOKENOMICS-V4.md` §4A.1): rewards are pro-rata to stake, so compounding preserves stake *shares*. Independent parties hold 1,099,570,620 / 18,146,400,000 = **6.05%** of the carryover; gate G1 requires ≥ 15% of circulating in independent hands. Therefore **G1 is unreachable by emission alone — not at year five, not at year forty. The only thing that moves it is coins changing hands.** If the founder abstains from staking entirely, the earliest G1 crossing under first-year emission plus unlocks is ≈ month 9 — a bound, not a forecast.

Verdict: FAIL, and it stays FAIL until the measured number changes. The protocol constrains the founder's *validator weight*; it does not and cannot constrain the founder's *holdings*. Any

framing of the three mechanisms above as fixing concentration would be false, and we do not offer one.

### 1.2 Mintable — what "fixed supply" is and is not, today (check 10)

Three layers, in decreasing strength:

1. **No privileged issuance path exists.** The transaction set is closed (`transition.rs:175–220`); the emission curve is a pure function that integrates to exactly the validator allocation and returns 0 after `EMISSION_SLOTS` (`tokenomics_v4.rs:406–421`); allocations sum to total supply by compile-time assertion (`tokenomics_v4.rs:217–232`).
2. **Divergence is caught indirectly.** A node minting off-curve computes a different `state_root` and forks itself off. Property tests pin the envelope (`tests/properties.rs:624`, `:665–681`).
3. **The decided direct invariant does not exist yet.** The decision of record (2026-08-12) is that every node refuses a block whose cumulative issuance would exceed the cap. We grepped for it: there is **no block-level cumulative-issuance check** in `tokenomics_v4.rs`, `transition.rs`, or `produce.rs`. This is an open implementation gap (§4, item 5).

The claim to make to an auditor, at its true strength and no stronger: *no mechanism inside the protocol can raise the supply* — no vote, no key, no governance path. A hard fork adopted by every operator can change any rule; "impossible to change" would be false and we do not claim it.

### 1.3 The grep behind checks 6, 11, 12, 13, 18, 23

The claim "no privileged control machinery" is testable, so we tested it: a crate-wide grep of `bloch-pos-committee/src/` for `blacklist|whitelist|allowlist|denylist|pause|freeze|owner|admin|privileged|governance|multisig|emeswitch|sudo|authority|master key|veto|council|override|ban|censor|upgrade` returns **no control mechanism**. Every hit is one of: doc prose (threat-model rationale), the *interface* freeze (a frozen API surface, not an account freeze), the node-local `ws.rs` checkpoint-conflict refusal that by test cannot override finality (`ws.rs:1112`), or the **retired taint set** — kept as inert, named-zero artifacts precisely so that any code still consulting it fails loudly (`staking.rs:172–183`, `:247`; `delegation.rs:124–130`; `tokenomics_v4.rs:106`; `state_root.rs:878`).

---

## 2. What CertiK would actually audit: the real surface

There is no contract to scan. The audit surface for Bloch is the node and consensus code. We propose the engagement cover these four crates, in this order:

| CRATE | ROLE | SOURCE LOC | TEST LOC | TESTS (MEASURED) | RESULT |
|---|---|---|---|---|---|
| `crates/bloch-pos-committee` | PoS consensus: state transition, staking, delegation, slashing, fork choice (LMD-GHOST), finality, fee market, tokenomics, state root | 15,614 | 4,088 | **334** | all pas<br>0 faile |

| CRATE | ROLE | SOURCE LOC | TEST LOC | TESTS (MEASURED) | RESULT |
|---|---|---|---|---|---|
| `crates/coherence-core` | Shielded pool (C1-frozen): SHAKE-256 commitments/nullifiers, commitment tree, sparse-Merkle nullifier set (C1.1, ratified), spend statement | 708 | 48 | **12** | all pas 0 faile |
| `crates/bloch-crypto` | Hybrid PQ signature suite: ML-DSA-65 ‖ Falcon-1024 AND-composition, suite dispatch, Falcon constant-time `clean` pin, addresses | 9,823 | 309 | **137** | all pas 0 faile (4 ignore — afte a one-line test-compi fix, se metho note |
| `crates/bloch-euvm` | eUTXO validator VM + Ustav (PSTRN-1) charter compiler + Kirpich fail-closed audit gate — **not consensus-wired**; the base other products build on | 10,766 | 1,808 | **331** | all pas 0 faile (4 ignore |

Method, so the numbers are reproducible: LOC is `wc -l` over `*.rs` under each crate's `src/` and `tests/`; test counts are the sum of `N passed` over every `test result:` line of `cargo test --release` run per crate on 2026-08-12 at commit `84ca42a`. We did **not** run a branch/line coverage instrument (tarpaulin/llvm-cov); the coverage statement we can make honestly is test count and pass rate, not percentage coverage.

One measurement finding, reported because an auditor would hit it on day one: at commit `84ca42a`, `cargo test` for `bloch-crypto` **did not compile** — the integration test `tests/tx_under_dual_and.rs` had three `Block` initializers stale against the `auxpow` field added to the struct (`src/core/mod.rs:1466`). The 137-test result above was measured after a three-line fix (`auxpow: None` in each initializer), committed alongside this dossier. The lesson stands regardless of the fix: integration-test targets in the Genesis-3 crates are not gating CI at present.

Context the auditor needs about the surface:

- `bloch-pos-committee` is a pure consensus crate: state in, state out, no I/O. The committed state is a closed 19-leaf list expressed twice — as a type (`state_root.rs:851-892`) and as tags (`state_root.rs:107-150`) — and the crate carries structural self-tests that an auditor should read first: `header.rs:626` (`single_derivation_path`, a source-scanning test that fails the build if block identity acquires a second derivation), `transition.rs:2518` (`every_committed_state_field_is_bound_by_the_root`), and `tests/one_state_root.rs` (the written statement of the one-derivation-path rule).
- `coherence-core` is small on purpose: it is the C1-frozen format layer. There is **no trusted setup** anywhere — the proof system is SP1 raw FRI-STARK (hash-based; the Groth16/Plonk wrappers are explicitly forbidden, `coherence-prover/README.md:44-53`), and the crate's only dependency is `sha3`.

- `bloch-crypto`'s highest-value review target is the one gate G7 already requires external review of: the online Falcon-1024 signing path (`docs/specs/BLOCH-FALCON-ONLINE-SIGNING.md`), including the symbol tripwire that proves the AVX2/AARCH64 non-constant-time variants are not linked (`crypto/mod.rs:492-529`) and the CI guard (`scripts/falcon-clean-guard.sh`).
- `bloch-euvm` is in scope not because it is consensus (it is not wired, and says so at `src/lib.rs:9-10`) but because Ustav-at-L1 is a decided direction (`docs/adr/ADR-040-evm-and-ustav-at-l1.md`) and two of the seven self-found findings (§3) live here — auditing it before promotion is the point of auditing it.

Supporting material the auditor receives with the code: the two PoS threat models (`BLOCH-POS-THREAT-MODEL.md`, `-2.md`), the frozen interface contract (`BLOCH-POS-INTERFACES.md`), the master migration design with the G1–G11 gates (`BLOCH-POS-SHA3-LATTICE-MIGRATION.md` §11), the honest gap inventory (`BLOCH-POS-GAPS.md`), the C1/C1.1 freeze docs, and the two prior internal audits (`docs/audit/AUDIT-2026-04-20_ERA1.md`, `docs/audit/groundstate_audit.md`). `docs/SECURITY_SELF_ASSESSMENT.md` states the prior-audit position plainly: **zero external audits to date**. This engagement would be the first.

---

## 3. Findings we found and fixed ourselves

Every item below was found by our own adversarial review, fixed, and pinned with a regression test — dates, commits, and test locations included, because an auditor who re-derives any of these should find our fix and our test already waiting.

**F-1. Two block identities (2026-08-11).** At integration, `interfaces` had a public tuple `BlockId` mintable from any 32 bytes, coexisting with the opaque `header::BlockId`; two disagreeing `BlockHeaderV4` types and three copies of the canonical header serialisation existed — 9 construction sites where the design allows 1. Fixed in `62ca5af` (dedup; `interfaces` re-exports `header`; genesis lost its literal id). The guard had been built in advance in `83cae4f`: `single_derivation_path` (`src/header.rs:626`) scans the crate source and fails on any second construction site, any manual trait impl, any alias, and any mention of the legacy `pow_hash`/`block_hash` identities. This class of bug caused a real Genesis-3 outage (block-DAG keyed by `pow_hash` while storage keyed by `block_hash`), which is why the guard is structural, not a code review note.

**F-2. The header did not commit to the body — in the code the node actually runs (2026-08-12).** `compute_post_state` never checked `body_root`, `attestation_root`, or `coherence_root`; those checks lived only in a parallel validator with no caller, and 178 green tests said nothing because the test builder stamped zeros. One `block_id` could name two different bodies. Fixed in `d29e3ad`: the transition now recomputes all three roots (step 3b), and `PosTransaction::canonical_bytes` was added because the body root was previously incomputable from typed state. Regression: `header_must_commit_to_body_attestations_and_coherence` (`src/transition.rs:2753`), asserting all three mismatch errors in both directions.

**F-3. Two state-root derivations (2026-08-12).** `transition` and `derive` committed *different* RANDAO leaf sets for the same block — two roots for one block, each seam green on its own tests; found independently by two agents the same day. Fixed in `4ca2646`: one

function, `state_root::randao_window` (`src/state_root.rs:924`), the stricter rule won. Regression: the five tests of `tests/one_state_root.rs` (reverting the old rule fails 2 of 5). The file opens with the written statement of the one-derivation-path rule.

**F-4. Slashing existed but nothing called it; then its bookkeeping was not under the root (2026-08-11/12).** First: `slashing.rs` was implemented, tested, sealed — and had no call site. Fixed in `8a3e0ea`: evidence became a transaction (`PosTransaction::SlashingEvidence`) validated inside the transition; invalid evidence rejects the whole block; 12 new tests including `evidence_transaction_slashes_operator_and_delegators_and_pays_whistleblower` (`src/transition.rs:2198`) and `replayed_evidence_rejects_the_second_block_even_swapped` (`:2372`). Second: the anti-replay set, correlation window, and delegator-loss ledger were not committed under the `state_root`, so a state-synced node could double-slash or reach a different verdict. Fixed in `319c7e6`: three new leaves (tags `0x11`–`0x13`, `src/state_root.rs:148–150`). Regression: `every_committed_state_field_is_bound_by_the_root` (`src/transition.rs:2518`) and `ejected_set_is_exactly_the_slashed_registry` (`:2815`).

**F-5. The Ustav supply cap was not a cap (HIGH, 2026-08-11).** `compile_supply` compared the cap against a value the *spender* wrote in the redeemer, never against the amount actually minted: with `cap = 1,000` an issuer minted 1,000,000,000 in one transaction. Since the program hash is the asset's policy id, promoting Ustav to consensus would have made the chain certify a false cap. Fixed in `0f67977`: the module reads the mint context and asserts `prior + delta <= cap`. Regressions: `supply_cap_cannot_be_bypassed_by_a_redeemer_supplied_amount` (`crates/bloch-euvm/src/modules.rs:725`), `supply_module_cap_is_enforced_over_total_supply` and `contrast_correct_cap_policy_rejects_the_same_over_cap_mint` (`tests/audit_modules_supply.rs:55`, `:135`).

**F-6. Redeemer padding bypass defeated the freeze control (CRITICAL, 2026-08-11).** The VM imposed no redeemer arity; compiled modules read the `frozen` flag at a fixed top-relative offset, so padding the redeemer with one extra value shifted every read onto attacker-controlled data — a frozen regulated-token output became spendable with no authority signature. Fixed in the same `0f67977`: a new `Op::ExpectDepth` opcode (`crates/bloch-euvm/src/lib.rs:136`, executed at `:379–383`) is emitted as the first op of every compiled module, making the expected arity part of the program and hence of `validator_hash` — the spender cannot renegotiate it. Regression: `transfer_policy_freeze_is_bypassed_by_padding_the_redeemer` (`tests/audit_modules.rs:55`), which deliberately keeps its bug-era name with the assertion inverted, plus an honest-arity-still-runs assertion.

**F-7. The shielded pool did not cross the genesis seam (2026-08-11), and its nullifier set had no canonical root (fixed 2026-08-12).** The Genesis-3→4 carryover pipeline moved only transparent `(addr, value)` balances; the ceremony stamped `coherence_root = [0u8; 32]` ("empty pool") with nothing carrying the tree or the nullifier set — a pool that does not cross as ordered leaves plus the complete nullifier set either burns every unspent note or revives every spent one. Worse, the header's coherence mirror was copied from the parent verbatim, validating nothing. Fixed in `eacddd9` (merged `c59a175`): the ceremony requires a fail-closed Coherence artifact, replays leaves in exact position order, carries the nullifier set whole, and "empty" became an attested artifact, never an assumption; the mirror now derives from committed state. Follow-up `c6fe0c1` ratified C1.1: the nullifier set is a sparse Merkle tree over

the 256-bit nullifier space in `coherence-core` (order-independent root, provable non-membership), removing the interim commitment the PoS crate had been computing on the pool's behalf. Regressions: `expected_coherence_derives_from_committed_state_not_the_parent_header` (`src/derive.rs:699`), `empty_pool_commits_a_real_root_never_the_zero_sentinel`, `leaf_order_is_consensus`, `dropping_a_nullifier_is_a_different_chain`, `shield_before_spend_after_crosses_the_seam` (`tools/genesis4-ceremony/src/lib.rs:1675-1735`), and the seven C1.1 set tests in `coherence-core/src/lib.rs:599-702`.

A pattern worth handing the auditor: F-2, F-3, and F-4b are one class — *a consensus value with two derivation paths, or none* — and the codebase now carries structural tests against the class itself (`single_derivation_path`, `one_state_root.rs`, `every_committed_state_field_is_bound_by_the_root`), not just the instances.

---

## 4. Open gaps — the list we would not want CertiK to discover first

1. **The PoS node is a devnet skeleton.** `crates/bloch-pos-node` runs real validators with real hybrid signatures over a localhost TCP mesh — and that is all. **No transactions** (`engine.rs:95` `NO_TXS`; non-empty bodies fail-closed rejected at `engine.rs:344-347`), **no libp2p** (`std::net::TcpListener` bound to `127.0.0.1`, `net.rs:147`), **no RPC** (no HTTP listener of any kind). The crate's own header lists the rest: no RocksDB store, no slashing-evidence pipeline, no weak-subjectivity sync, no fork choice beyond a linear chain, no mainnet genesis manifest (`main.rs:10-15`). The consensus *crate* is substantially built; the *node* around it is 2,313 lines of scaffolding.
2. **G1–G11 are defined and none is measured.** The go/no-go gates (`BLOCH-POS-SHA3-LATTICE-MIGRATION.md` §11) attach to a transition height that is only reached after a hybrid phase that has not started; Genesis-4 has not launched, so no gate has an observed value. G1 additionally has an arithmetic proof that it cannot be reached by emission alone (§1.1). The measurement functions for G2/G3 exist in code (`delegation.rs:279-307`) but their semantics are specified in no document (`BLOCH-POS-GAPS.md` §4 item 1).
3. **Genesis keys do not exist.** `BLOCH-GENESIS-KEYS.md` is explicit: "NO production key exists yet." The custody plan (air-gap, sharding, exposure windows, T–2-week validator keygen) is DRAFT. The carryover exception is named in the doc itself: the Genesis-3 keys guarding the 3.77 B liquid carryover already exist, were generated long ago under unknown conditions, and sit outside the plan, inside the risk.
4. **EVM at L1 is a design.** Only the state-root leaf (`TAG_EVM_COMMITMENT`, `state_root.rs:136`) is implemented. There is no execution crate, no deposit/withdrawal transaction kinds, no gas limits fixed, and — decisive for an auditor — **the authorisation model (secp256k1 vs PQ-only vs dual) is an undecided founder decision** (`BLOCH-L1-EVM-AUTHORIZATION.md`, recommendation recorded, nothing ratified). Ustav/Kirpich promotion to L1 is likewise a proposal with no code (`BLOCH-USTAV-L1.md`; `bloch-euvm` is not consensus-wired).
5. **The decided supply-cap consensus invariant is unimplemented** (§1.2): no block-level check rejects over-cap cumulative issuance; the cap holds by construction and by state-

root divergence only.

6. **Fee-era boundary wiring is incomplete.** `distribute_producer_fees` (`fee_market.rs:267–282`) — the fix that lets delegators keep earning once fees are the whole budget — is implemented and tested but **not yet called from the transition**, which still credits the raw producer share (`fee_market.rs:261` says so itself).
7. **SPDX headers are not universal.** 59 of 115 `.rs` files under `crates/` lack the per-file SPDX line — including all seven `bloch-euvm` source files and `coherence-core/src/lib.rs`. Manifests all declare AGPL-3.0-or-later (two deliberate vendored exceptions); `bloch-pos-committee` and `bloch-pos-node` are 100% clean.
8. **Repository publication is asserted, not scheduled** (check 20): the repo is private, and no document commits to a date, URL, or process for opening it, while ADR-033 rests its decentralisation claim on "open-sourced before launch".
9. **Zero external audits to date** (`SECURITY_SELF_ASSESSMENT.md`), one primary developer, and a fleet operated by the founder. The prior internal audits (ERA-1, GroundState) cover a predecessor codebase.
10. **Known spec-vs-code drift is inventoried** in `BLOCH-POS-GAPS.md` — including the `CapStatus::Deferred` consensus behaviour absent from the spec, the stale §7A tables in the tokenomics doc, and the frozen interface amended by the EVM leaf. We hand the auditor that inventory rather than a claim of alignment.

---

## 5. Contradictions flagged while preparing this dossier

These are internal inconsistencies between decisions of record, documents, and code, found during preparation. They must be resolved before the dossier is sent, or sent with this section intact — an auditor will find them within hours.

1. **Total supply: 21 B in code and docs vs 100 B decision of record.** `tokenomics_v4.rs:33` pins `TOTAL_SUPPLY_BLOCH = 21_000_000_000`, with a doc comment recording the founder's 2026-08-11 decision as a *revert* of a 100 B draft; `BLOCH-TOKENOMICS-V4.md` §1 says the same. The 2026-08-12 decision of record reverses this again: 100 B as a pure split (×4.7619, every percentage unchanged — a redenomination, not "more supply for holders"). **The redenomination as specified cannot land in the current representation**: at 8 decimal places, 100 B is $1.0 \times 10^{19}$ sat, which violates both compile-time assertions at `tokenomics_v4.rs:254–255` (the 8× `u64` headroom bound and the `i64::MAX` bound that exists because the Go SDK's `Satoshis` is a signed int64 — the very overflow the 21 B revert was recorded as fixing). Landing 100 B requires reducing decimal places, changing the satoshi representation, or accepting the loss of headroom — a decision, not a find-and-replace. Concentration percentages in §1.1 are denomination-independent and unaffected either way.
2. **Genesis-3 terminal height: 50,000 decided, 50,000 everywhere.** The 2026-08-12 decision lowers the halt to height 39,918 (~4 days out at decision time). Every committed artifact says 50,000 — `BLOCH-TOKENOMICS-V4.md` §3.1/§3.2, the migration spec, the node-integration plan, the interfaces doc, and five public portal pages; the value 50,000 appears in no height context anywhere in `docs/`. §3.1's notice-period argument was also written for

50,000. The chain stopped at 39,918 before reaching, the snapshot height in this dossier's §1.1 measurements should be treated as "the terminal height", not a number.

3. **Validator bond.** The decision of record sets the bond near 25,000 BLCH (Ethereum's fraction of supply under 100 B), down from 100,000. This follows the supply decision and inherits contradiction 1. It widens who *may* validate; it does nothing about who *does*, and this dossier does not describe it as a concentration fix.

---

## 6. What this dossier does not claim

- It does not claim the scanner's checklist was "passed". Two applicable checks fail today (9, 20), and the substitutes for three more carry open caveats (10, 14, 22).
- It does not claim percentage test coverage — not measured, only test counts and pass rates (§2 method note).
- It does not claim the supply cap is unchangeable — only that no mechanism inside the protocol can change it, which is the strongest true statement.
- It does not claim decentralisation. The chain starts centralised, by construction; the gates measure the distance from there, and none has been measured.

---

## Annex A — document inventory for the engagement

| DOCUMENT | ONE LINE |
|---|---|
| `docs/specs/BLOCH-POS-SHA3-LATTICE-MIGRATION.md` | Master Genesis-4 PoS design; §11 = G1–G11 gates. DRAFT. |
| `docs/specs/BLOCH-TOKENOMICS-V4.md` | Supply, allocation, §4A/§4A.1 concentration analysis. DRAFT, parameters not frozen. |
| `docs/specs/BLOCH-POS-INTERFACES.md` | Frozen consensus interface contract (with recorded amendments). |
| `docs/specs/BLOCH-POS-GAPS.md` | The honest implemented/specified/neither inventory. |
| `docs/specs/BLOCH-POS-THREAT-MODEL.md`, `-2.md` | Two adversarial passes over the PoS design. |
| `docs/specs/COHERENCE-C1.md`, `COHERENCE-C1.1.md` | Shielded-pool format freeze; C1.1 is the only RATIFIED spec. |
| `docs/specs/COHERENCE-G11-SHADOW-FORKS.md` | G11 acceptance-evidence plan (three shadow forks). |
| `docs/specs/BLOCH-FALCON-ONLINE-SIGNING.md` | The path G7 requires externally reviewed — a natural CertiK work item. |
| `docs/specs/BLOCH-GENESIS-KEYS.md` | Key ceremony and custody plan. DRAFT; no production key exists. |
| `docs/specs/BLOCH-ENTITY-STRUCTURE.md` | Two-entity governance structure. DRAFT, jurisdiction open. |
| `docs/specs/BLOCH-L1-EVM-*.md` (5 docs) | EVM-at-L1 design set; authorisation undecided. |
| `docs/specs/BLOCH-USTAV-L1.md`, `BLOCH-KIRPICH-UNDER-POS.md` | Charter-at-consensus proposals; no code. |
| `docs/adr/` (ADR-023…040) | Decision record: governance, custody, licence (ADR-039), EVM/Ustav at L1 (ADR-040), churn |

| DOCUMENT | ONE LINE |
|---|---|
| | (ADR-038), ownerless-claim retraction (ADR-036). |
| `docs/audit/AUDIT-2026-04-20_ERA1.md`, `docs/audit/groundstate_audit.md` | Prior internal audits of the predecessor codebase. |
| `docs/SECURITY_SELF_ASSESSMENT.md` | Self-assessment vs Bitcoin Core; records zero external audits. |
| `audit/CONSOLIDATED-SECURITY-REPORT.md` | Roll-up of every internal security review to date. |
| `docs/INTERNAL-AUDIT-PLAN.md` | The internal review plan that names CertiK among candidate firms. |
| `REPRO.md`, `repro-manifest.sh`, `scripts/falcon-clean-guard.sh`, `deny.toml`, `audit.toml` | Build-integrity and supply-chain tooling. |

> **Correction, 2026-08-13.** This edition was written while the halt was still ahead. Genesis-3 did not reach 50,000: production was stopped at height **39,918** and the terminal snapshot was taken there — 452,726 outputs across 16 addresses, 3,810,744,000 BLOCH, set root `7c756ee8…`, produced independently on two nodes with byte-identical results. Every carryover figure in this document is measured from that file. It also stopped **82 blocks before** the Emission V3 flag day at height 40,000, so the block reward ended at 8,400 BLOCH and never became 2,600. The narrative below describing the 80,000 → 50,000 decision is left as written: it is what happened, and a document that edits its own history to match the outcome is worth less than one that does not.